

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ КАФЕДРА
ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

ПОЯСНЮВАЛЬНА ЗАПИСКА

до кваліфікаційної роботи освітнього ступеня «бакалавр» за
спеціальністю 121 «Інженерія програмного забезпечення» (освітня
програма «Інженерія програмного забезпечення»)

на тему:

«Мобільний додаток для вивчення іноземних мов»

Виконав студент групи ІПЗ-22-3
СОКУЛЬСЬКИЙ Олег Ігорович

Керівник роботи:
БАКАЛЮК Тетяна Анатоліївна

Рецензент:
ЛОБАНЧИКОВА Надія Михайлівна

ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
Факультет інформаційно-комп'ютерних технологій
Кафедра інженерії програмного забезпечення
Спеціальність 121 «Інженерія програмного забезпечення»

«ЗАТВЕРДЖУЮ»

Завідувач кафедри ІПЗ

_____ Тетяна ВАКАЛЮК

«___» _____ 202_ р.

ЗАВДАННЯ
НА КВАЛІФІКАЦІЙНУ РОБОТУ

Здобувач вищої освіти: **СОКУЛЬСЬКИЙ Олег Ігорович**

Керівник роботи: **ВАКАЛЮК Тетяна Анатоліївна**

Тема роботи: **«Мобільний додаток для вивчення іноземних мов»**,

затверджена Наказом закладу вищої освіти від «_____» _____ 2026 р., №___

Вихідні дані для роботи: **об'єкт дослідження – процес цифрової взаємодії користувача з освітнім контентом, що включає механізми**

засвоєння лексичного матеріалу, систематизацію знань та

персоналізацію навчання через мобільні інтерфейси.; предмет

дослідження – архітектурні методи побудови мобільного застосунку,

алгоритми адаптивного тестування користувачів та технологічний стек

, що забезпечує стабільну роботу сервісу з вивчення мов.

Консультанти з бакалаврської кваліфікаційної роботи із зазначенням розділів, що їх стосуються:

Розділ	Прізвище, ініціали та посади консультанта	Підпис, дата	
		завдання видав	завдання прийняв
1	Вакалюк Т.А.		
2	Вакалюк Т.А.		
3	Вакалюк Т.А.		

РЕФЕРАТ

Пояснювальна записка до бакалаврської кваліфікаційної роботи на тему «Мобільний додаток для вивчення іноземних мов» викладена на XX сторінках друкованого тексту. Візуальний склад роботи включає XX рисунків та XX таблиць. Список використаної літератури охоплює XX джерел, додатки розміщені на XX сторінках. Повний обсяг пояснювальної записки – XX сторінок.

Об'єкт дослідження – архітектурні принципи та технологічні цикли розробки мобільного програмного забезпечення, спрямованого на автоматизацію освітніх процесів.

Предмет дослідження – інструментарій, алгоритмічні рішення та методології побудови мобільного сервісу для засвоєння іноземної лексики та граматики.

Мета роботи – проектування та реалізація функціонального мобільного застосунку, який забезпечує інтерактивну взаємодію користувача з навчальними матеріалами, зберігання прогресу навчання та персоналізацію освітнього треку.

Методологічна база. У ході виконання роботи було застосовано системний аналіз для визначення користувацьких потреб, порівняльне дослідження існуючих EdTech-рішень.

Практична цінність. Результатом роботи є робочий прототип мобільного застосунку, адаптованого під потреби сучасних користувачів. Проект реалізовано з використанням сучасної мобільної розробки (наприклад, кросплатформеного підходу), що дозволяє досягти високої продуктивності та нативного досвіду взаємодії. повторень.

КЛЮЧОВІ СЛОВА: МОБІЛЬНИЙ ДОДАТОК, ІНОЗЕМНІ МОВИ, EDTECH, ІНТЕРФЕЙС КОРИСТУВАЧА, КРОСПЛАТФОРМЕНА РОЗРОБКА, АДАПТИВНЕ НАВЧАННЯ, ПЕРСОНАЛІЗАЦІЯ.

					ДУ «Житомирська політехніка».26.121.23.000 - ПЗ				
Змн.	Арк.	№ докум.	Підпис	Дата					
Розроб.		Сокульський О.І.			Мобільний додаток для вивчення іноземних мов	Лім.	Арк.	Аркушів	
Перевір.								4	59
Керівник		Вакалюк Т.А.				ФІКТ, Гр. ІПЗ-22-3			
Н. контр.									
Зав. каф.		Вакалюк Т.А.							

					ДУ «Житомирська політехніка».26.121.23.000 - ПЗ		
Змн.	Арк.	№ докум.	Підпис	Дата	Мобільний додаток для вивчення іноземних мов		
Розроб.		Сокульський О.І.					
Перевір.							
Керівник		Вакалюк Т.А.					
Н. контр.							
Зав. каф.		Вакалюк Т.А.			ФІКТ, Гр. ІПЗ-22-3		
					Літ.	Арк.	Аркушів
						4	59

ЗМІСТ

РЕФЕРАТ.....	3
ЗМІСТ.....	5
ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ	6
ВСТУП.....	7
РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ.....	10
1.1 Постановка задачі та огляд існуючих систем-аналогів	10
1.2 Функціональні та нефункціональні вимоги до системи	15
1.3 Вибір і обґрунтування архітектурного рішення.....	18
1.4 Вибір стеку технологій та апаратного забезпечення.....	20
Висновки до першого розділу	23
РОЗДІЛ 2. ПРОЄКТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ	25
2.1 Моделювання варіантів використання та поведінки системи	25
2.2 Проєктування логічної структури бази даних	31
2.3 Проєктування компонентної структури та взаємодії модулів	39
2.4 Розробка основних алгоритмів роботи платформи.....	51
Висновки до другого розділу	58

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

БД – база даних

ПЗ – програмне забезпечення

СУБД – система управління базами даних

API (Application Programming Interface) – прикладний програмний інтерфейс

DTO (Data Transfer Object) – об'єкт передачі даних між шарами архітектури.

CRUD (Create, Read, Update, Delete) – базові функції управління даними (створення, читання, оновлення, видалення)

JSON (JavaScript Object Notation) – текстовий формат обміну даними

JWT (JSON Web Token) – відкритий стандарт для безпечної передачі інформації у вигляді JSON-об'єкта

MVVM (Model-View-ViewModel) – архітектурний патерн, що є стандартом для сучасної мобільної розробки

ORM (Object-Relational Mapping) – технологія програмування, яка зв'язує бази даних з об'єктно-орієнтованим кодом

RBAC (Role-Based Access Control) – управління доступом на основі ролей

REST (Representational State Transfer) – архітектурний стиль взаємодії компонентів розподіленого застосунку в мережі

SQL (Structured Query Language) – мова структурованих запитів до реляційних баз даних

UI (User Interface) – інтерфейс користувача

SDK (Software Development Kit) – набір засобів розробки для певної платформи (Android/iOS).

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

ВСТУП

Актуальність теми. Трансформація освітньої галузі в умовах глобальної цифровізації зумовлює перехід від класичних методів навчання до використання персоналізованих мобільних інструментів. Сучасний темп життя вимагає від користувачів гнучкості, що робить мобільні застосунки (EdTech-рішення) пріоритетним засобом вивчення іноземних мов. Проте, попри велику кількість існуючих продуктів, значна частина з них має обмеження: або надмірну складність інтерфейсу, або відсутність адаптивних алгоритмів, що враховують індивідуальну швидкість засвоєння матеріалу.

Традиційні підходи, орієнтовані на статичний контент, поступово втрачають ефективність. Користувачам потрібні інструменти, що забезпечують безперервний доступ до знань, ігрові механіки (гейміфікацію) та можливість відстеження прогресу в реальному часі. Розробка програмних систем такого типу є складним інженерним завданням, оскільки вимагає не лише створення зручного GUI, а й проєктування надійної архітектури, здатної синхронізувати локальні дані користувача з хмарними сервісами та забезпечувати високу швидкість відгуку інтерфейсу.

З огляду на це, проєктування та реалізація мобільного застосунку для вивчення іноземних мов, побудованого на сучасних архітектурних патернах (зокрема MVVM) та хмарних технологіях, є актуальним науково-практичним завданням для інженерії програмного забезпечення.

Об'єкт дослідження – процеси проєктування, розробки та функціонування мобільних інформаційних систем для інтерактивного навчання та управління освітнім контентом.

Предмет дослідження – методи, архітектурні шаблони та програмні інструменти розробки кросплатформених мобільних додатків для опанування іноземної лексики.

Мета роботи полягає у проєктуванні та розробці мобільного застосунку для вивчення іноземних мов, який автоматизує процес подачі навчального

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

матеріалу, забезпечує контроль знань через систему тестування та пропонує персоналізований досвід взаємодії.

Для досягнення поставленої мети було сформульовано та вирішено такі завдання:

Проаналізувати предметну область EdTech, дослідити психолого-педагогічні аспекти цифрового навчання та виконати порівняльний огляд існуючих мобільних рішень.

Сформулювати функціональні та технічні вимоги до системи, обґрунтувати вибір технологічного стеку для мобільної розробки (напр. Flutter, React Native або Native) та обрати засоби зберігання даних.

Спроектувати архітектуру застосунку, визначивши логіку взаємодії між компонентами на основі сучасних патернів проєктування.

Розробити структуру бази даних для збереження словникових запасів, профілів користувачів та результатів тестування, а також спроектувати API для обміну даними.

Реалізувати програмний продукт, включаючи клієнтську частину з інтерактивними вправами, модулі гейміфікації та систему візуалізації навчального прогресу.

Провести верифікацію та тестування функціоналу, оцінити продуктивність інтерфейсу та зручність користувацького досвіду (UX).

Методи дослідження. Для розв'язання поставлених завдань у роботі застосовано комплекс методів системного аналізу (для дослідження ринку EdTech та формалізації вимог), методи об'єктно-орієнтованого аналізу та моделювання UML (для побудови логічної структури додатка), а також методи теорії графів та інтервальних повторень (для розробки алгоритмів засвоєння слів). Використано методи проєктування реляційних баз даних для забезпечення цілісності навчального контенту.

Практичне значення одержаних результатів. Створений мобільний застосунок надає користувачам персоналізований інструментарій для вивчення іноземних мов, що значно знижує поріг входу в навчальний процес та підвищує

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				8
Змн.	Арк.	№ докум.	Підпис	Дата		

рівень запам'ятовування нової лексики. Архітектурна гнучкість системи, побудована на принципах Clean Architecture та патерні MVVM, дозволяє масштабувати продукт додавати нові мовні пари, впроваджувати ігрові модулі або інтегрувати зовнішні словникові API без рефакторингу основної логіки.

		Сокульський О.І.			ДУ «Житомирська політехніка».26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				9
Змн.	Арк.	№ докум.	Підпис	Дата		

РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ

1.1 Постановка задачі та огляд існуючих систем-аналогів

Аналіз предметної області та проблематики

Предметною областю дипломного проєкту є сфера мобільних освітніх технологій (EdTech), зокрема сегмент програмного забезпечення для вивчення іноземних мов. В умовах глобалізації та переходу до концепції "Lifelong Learning" (навчання впродовж життя), володіння іноземними мовами стало базовою компетенцією. Проте ефективність опанування нових знань безпосередньо залежить від інструментарію, який використовує учень. Традиційні підходи та застарілі цифрові рішення стикаються з низкою критичних проблем.

Класичний процес вивчення мови (за підручниками або статичними PDF-матеріалами) є фрагментарним і часто призводить до втрати мотивації. Основні перешкоди включають:

Відсутність персоналізації: універсальні програми не враховують індивідуальний темп забування інформації.

Складність систематизації: самостійний збір лексики, ведення паперових словників та ручне створення карток для повторення вимагає величезних часових ресурсів.

Проблема "кривої забування": без алгоритмічного контролю за інтервалами повторень більша частина вивченого матеріалу втрачається вже за кілька днів.

Усвідомлення цих бар'єрів призвело до появи інтелектуальних мобільних платформ. Основна ідея сучасного підходу — перекласти рутинні процеси (планування повторень, підбір вправ, перевірку помилок) з людини на програмні алгоритми. Замість пасивного читання користувач отримує інтерактивне середовище, де мобільний додаток виступає як персональний тьютор.

Важливою вимогою до розробки є забезпечення високої швидкодії та безперебійної роботи в офлайн-режимі. Мобільний контекст використання передбачає короткі навчальні сесії ("на ходу"), тому додаток повинен мати оптимізовану архітектуру (наприклад, патерн MVVM) для миттєвого відгуку

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				10
Змн.	Арк.	№ докум.	Підпис	Дата		

інтерфейсу. Система має бути адаптивною: коректно відображати навчальний контент на різних типах пристроїв — від бюджетних смартфонів до планшетів з високою роздільною здатністю.

Ключовим рівнем автоматизації є управління навчальним контентом. Додаток повинен реалізовувати принцип реактивної взаємодії: зміна рівня знань користувача має миттєво впливати на складність наступних вправ. Окрім візуальної частини, система має автоматизувати роботу з динамічними сутностями словниковими базами, аудіофайлами та прогресом користувача.

Важливим інженерним аспектом є впровадження механізмів data binding (прив'язки даних). Оновлення статусу вивчення слова в локальній базі даних (наприклад, через SQLite або Room) має автоматично синхронізуватися з візуальними індикаторами прогресу, що виключає необхідність ручного оновлення сторінок інтерфейсу.

Опираючись на проведений аналіз, головним завданням роботи є проєктування та розробка програмного продукту «Мобільний додаток для вивчення іноземних мов». Система має стати комплексним рішенням, яке дозволяє користувачу без зайвих зусиль формувати індивідуальний словниковий запас, проходити інтерактивні тренування та відстежувати свій розвиток через наочну статистику.

Досягнення цієї мети вимагає глибокого дослідження архітектурних підходів до мобільної розробки, методів оптимізації локальних БД та реалізації алгоритмів адаптивного навчання, що відповідає сучасним стандартам інженерії програмного забезпечення.

Дослідження архітектурних підходів до побудови мобільних інтерфейсів та візуалізації навчання

Сучасний розвиток мобільного програмного забезпечення та зростання вимог до персоналізації освітнього процесу зумовили перехід до концепції адаптивних інтерфейсів. У контексті застосунків для вивчення мов ця парадигма вимагає відходу від статичної подачі матеріалу. З інженерної точки зору це означає, що рутинні процеси — такі як управління станом екрана,

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				11
Змн.	Арк.	№ докум.	Підпис	Дата		

обробка життєвого циклу компонентів та транзакції до локальних баз даних — мають бути інкапсульовані всередині архітектурних шарів платформи.

При проєктуванні мобільних систем, що передбачають високу залученість користувача, фундаментальним є вибір підходу до формування інтерфейсу. Аналіз існуючих рішень дозволяє виділити два ключові методи: імперативний та декларативний.

Декларативний підхід (реалізований у сучасних фреймворках, таких як Jetpack Compose для Android або SwiftUI для iOS) ідеально узгоджується з концепцією побудови мовних додатків. Основна увага приділяється опису кінцевого стану UI залежно від даних. На практиці це дозволяє розглядати інтерфейс як набір незалежних ізольованих компонентів (карток слів, прогрес-барів, модулів тестування), що динамічно змінюються залежно від успіхів учня.

Для вирішення проблеми жорсткої фіксації структури екрана у вихідному коді доцільно застосувати концепцію Server-Driven UI (інтерфейс, керований сервером). Цей патерн дозволяє динамічно змінювати склад навчальних модулів без необхідності повного оновлення застосунку через стори (App Store/Google Play). Конфігурація уроку або тесту передається з сервера у форматі JSON, що містить опис типів вправ, посилання на медіафайли та логіку перевірки відповідей.

Після отримання такої схеми мобільний клієнт динамічно рендерить відповідні UI-компоненти. Для збереження візуальної цілісності архітектура системи має спиратися на принципи методології Atomic Design.

Така структура гарантує стабільність відображення навчального контенту на пристроях з різною діагоналлю екрана та забезпечує високу швидкість відгуку (фреймрейт), що є критичним для позитивного користувацького досвіду.

Аналіз існуючих систем-аналогів

Ринок EdTech-рішень для мобільного навчання демонструє стабільне зростання. Користувачі шукають інструменти, що дозволяють ефективно використовувати короткі проміжки часу для навчання. Для формування вимог

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				12
Змн.	Арк.	№ докум.	Підпис	Дата		

до власної системи було проведено аналіз двох найбільш репрезентативних рішень: Duolingo та Anki.

Платформа Duolingo

Це лідер сегмента, що базується на тотальній гейміфікації. Головна перевага низький поріг входу та підтримка мотивації через ігрові механіки (ліги, ударні режими, нагороди). З інженерної точки зору Duolingo має складну серверну логіку для керування контентом. Проте основним недоліком для досвідчених користувачів є жорстка лінійна структура. Користувач не може самостійно формувати списки слів для вивчення, імпортувати власні словникові бази або фокусуватися виключно на специфічній термінології. Система працює як «чорна скринька» з закритим алгоритмом подачі матеріалу.

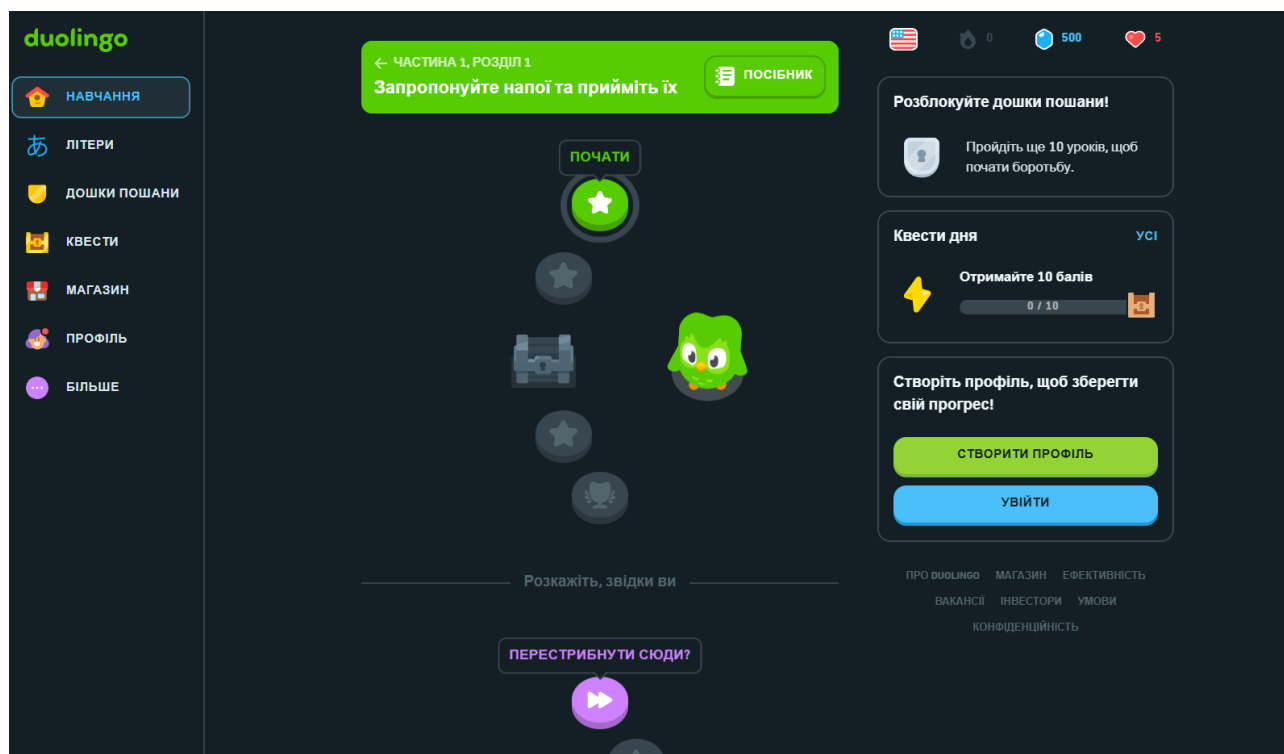


Рис. 1.1 – Інтерфейс вебзастосунку Duolingo

Платформа Anki.

Це професійний інструмент, що базується на методі інтервальних повторень (Spaced Repetition). Його перевага — повна свобода у створенні контенту та відкритість алгоритмів. З точки зору UX/UI, Anki має застарілий інтерфейс та високу складність налаштування для пересічного користувача. Платформа не пропонує інтерактивних вправ (лише картки «питання-відповідь») і не має

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				
Змн.	Арк.	№ докум.	Підпис	Дата		13

сучасних механізмів візуалізації прогресу, що знижує залученість.

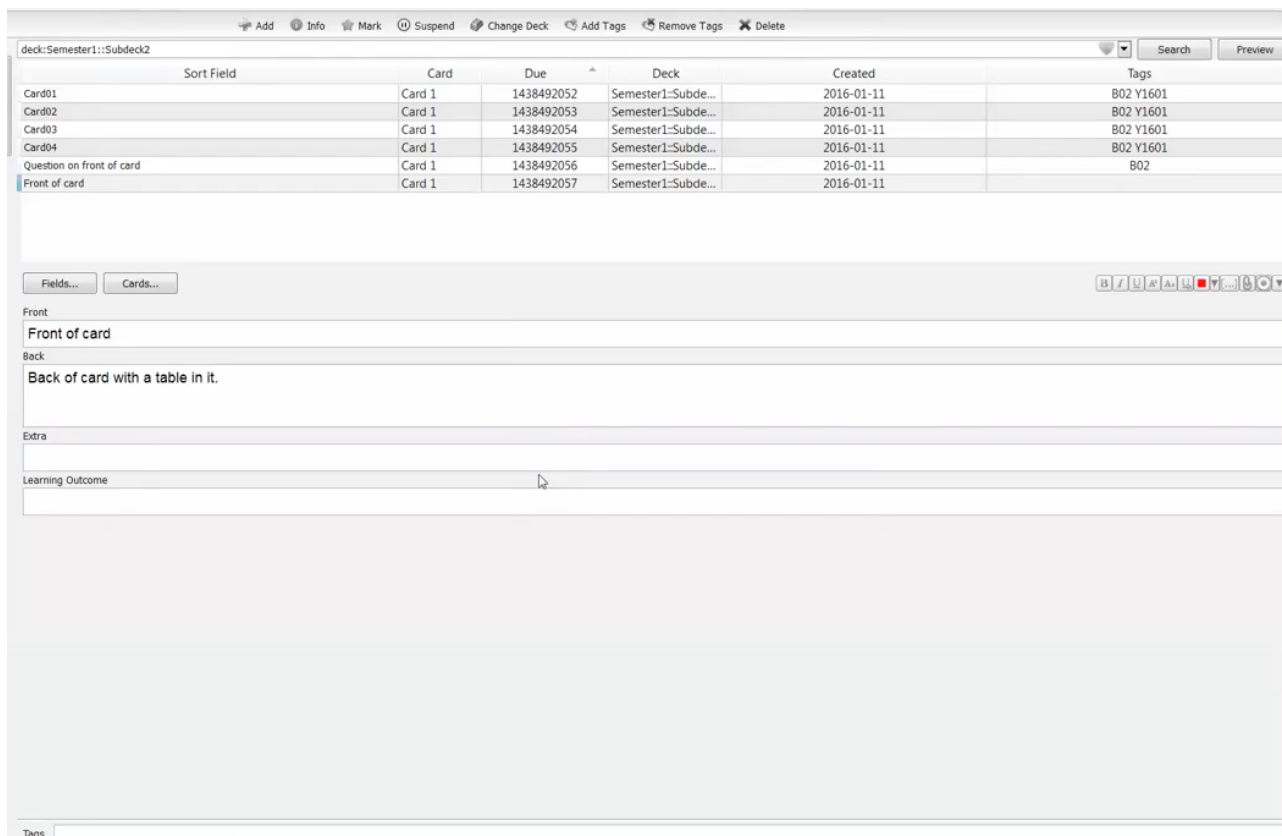


Рис. 1.2 – Інтерфейс вебзастосунку Anki

Порівняльна характеристика систем

Для обґрунтування функціональних вимог до розроблюваного застосунку, порівняльні критерії зведено у таблицю 1.1.

Таблиця 1.1 – Порівняння інформаційних систем спортивної аналітики

Критерії порівняння	Duolingo	Anki	Додаток, що розробляється
Гейміфікація та ігрові механіки	Так	Ні	Так
Створення власних наборів слів	Ні	Так	Так
Алгоритм інтервальних повторень	Так (прихований)	Так (налаштовуваний)	Так
Офлайн-режим роботи	Обмежено	Так	Так
Тип інтерфейсу	Жорстко лінійний	Картковий	Модульний (адаптивний)
Архітектурний підхід	Клієнт-сервер	Локальна БД	MVVM + Cloud Sync

Інтерактивні вправи (не лише картки)	Так	Ні	Так
Можливість імпорту зовнішніх словників	Ні	Так	Так

Висновки з аналізу аналогів

Проведений аналіз показує, що існуючі рішення або занадто жорсткі в плані контенту (Duolingo), або мають застарілий інтерфейс та складну логіку налаштування (Anki).

Жодна з платформ не забезпечує ідеального балансу між простотою ігрового додатка та гнучкістю професійного інструмента для створення власних освітніх траєкторій. Це підтверджує актуальність розробки власного мобільного застосунку, який поєднає сучасну архітектуру (MVVM) для забезпечення високої швидкодії. Гнучкість контенту, що дозволить користувачу самостійно керувати базою знань. Інтуїтивний UI, адаптований під методики інтенсивного запам'ятовування.

1.2 Функціональні та нефункціональні вимоги до системи

Аналіз цільової аудиторії та основних сценаріїв використання системи

Визначення цільової аудиторії є фундаментом для проєктування архітектури мобільного застосунку. Розуміння потреб користувачів дозволяє оптимізувати мобільний інтерфейс (UI) та забезпечити позитивний досвід взаємодії (UX). Аналіз сегмента EdTech дозволяє виділити три групи користувачів, кожна з яких має специфічні запити до програмного продукту.

Перший сегмент – активні здобувачі освіти (студенти, школярі). Це користувачі, для яких критично важливою є швидкість засвоєння матеріалу в умовах обмеженого часу. Їхня основна проблема – складність утримання великих обсягів лексики без систематичного повторення. Мобільний додаток вирішує це завдання через механізми гейміфікації та push-сповіщень. Користувач отримує доступ до інтерактивних карток та коротких сесій навчання, що дозволяє інтегрувати процес вивчення мови у повсякденний графік.

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				15
Змн.	Арк.	№ докум.	Підпис	Дата		

Другий сегмент фахівці, що вивчають професійну лексику (ESP English for Specific Purposes). Це професіонали (розробники, лікарі, юристи), яким не потрібні загальні курси, а необхідна гнучкість у формуванні власної бази знань. Головна проблема існуючих аналогів закритість контенту. Розроблюваний застосунок пропонує інструментарій для кастомізації: користувач може самостійно створювати тематичні модулі, імпортувати власні списки слів та налаштовувати пріоритетність вивчення термінів.

Третій сегмент адміністратори та розробники контенту. Користувачі, які відповідають за наповнення глобальних баз даних та підтримку актуальності навчальних матеріалів. Для них система реалізує принцип централізованого джерела істини (Single Source of Truth). Це означає, що будь-які виправлення у словниковій базі на сервері автоматично синхронізуються з усіма мобільними клієнтами, гарантуючи достовірність навчального контенту.

Основні сценарії використання

На основі аналізу аудиторії сформовано базові сценарії взаємодії, які лягли в основу проєктування логіки застосунку.

Сценарій 1: Адаптивне тренування (Daily Review) Користувач запускає застосунок, і система на основі алгоритму інтервальних повторень автоматично формує чергу слів, які потребують закріплення. Сценарій включає проходження серії інтерактивних вправ (вибір перекладу, аудіювання, написання). Після завершення сесії мобільний клієнт реактивно оновлює локальну базу даних (SQLite/Room), перераховуючи дату наступного повторення для кожної сутності, та синхронізує прогрес із хмарою.

Сценарій 2: Створення персоналізованого модуля Користувач ініціює створення нової групи слів. Він вводить нові лексичні одиниці, а система, використовуючи сторонні API (наприклад, Oxford Dictionary або Google Translate), автоматично пропонує варіанти перекладу та транскрипції. Після збереження система генерує нову JSON-схему для даного модуля, що робить його доступним для проходження в режимі офлайн.

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				16
Змн.	Арк.	№ докум.	Підпис	Дата		

Сценарій 3: Аналіз прогресу та експорт даних Користувач переходить у розділ статистики для візуальної оцінки динаміки навчання. Система буде графіки на основі накопичених даних про кількість вивчених слів та успішність тестувань. За потреби користувач може ініціювати процедуру експорту свого словникового запасу у форматі CSV або JSON для використання в інших освітніх інструментах, що забезпечує відкритість та мобільність даних.

Функціональні вимоги до системи

Функціональні вимоги визначають конкретний перелік можливостей, які реалізує мобільний застосунок для задоволення потреб користувача. Архітектура системи базується на моделі авторизації та розмежування прав доступу, що охоплює життєвий цикл освітнього контенту.

Детальний перелік функціональних можливостей за ролями наведено у таблиці 1.2.

Таблиця 1.2 – Функціональні вимоги до мобільного застосунку

Роль користувача	Функціональні можливості у системі
Гість (неавторизований користувач)	Перегляд ознайомчого екрана (Onboarding). Ознайомлення з доступними мовними курсами. Реєстрація нового акаунта, авторизація через Email/Password або OAuth (Google/Apple). Відновлення пароля.
Користувач (Учень)	Управління профілем та налаштування цілей навчання. Створення персональних словникових модулів. Проходження інтерактивних уроків та тестів. Використання режиму інтервальних повторень.
Контент-менеджер	Управління глобальною бібліотекою слів та фраз (CRUD). Модерація готових навчальних планів. Завантаження медіафайлів (аудіо-вимова, ілюстрації). Формування шаблонів вправ та тестів через адміністративну панель.
Адміністратор	Повний контроль над системою та базою користувачів. Призначення ролей. Аналітика загальної активності аудиторії. Налаштування параметрів безпеки та керування API-ключами зовнішніх сервісів перекладу.

Нефункціональні вимоги

Нефункціональні вимоги визначають якісні характеристики роботи застосунку, такі як стабільність, швидкість та безпека, що є критично важливими для мобільних платформ.

Вимоги до продуктивності та швидкодії

Оскільки застосунок орієнтований на мобільне використання, критичним є час холодного запуску — він не повинен перевищувати 2 секунд. Час відгуку інтерфейсу на дії користувача (кліки, перемикання екранів) має бути миттєвим (до 100 мс). Рендеринг анімацій у навчальних модулях повинен відбуватися стабільно при 60 FPS, щоб забезпечити плавність UX на пристроях з різною потужністю.

Вимоги до надійності та відмовостійкості

Однією з ключових вимог є підтримка офлайн-режиму. Застосунок повинен дозволяти проходити завантажені уроки без доступу до мережі, синхронізуючи результати з сервером при відновленні з'єднання. Для економії трафіку та заряду акумулятора передача даних має здійснюватися у стисненому форматі JSON, а кешування медіаконтенту бути пріоритетним.

Вимоги до безпеки

Система повинна гарантувати цілісність даних користувача при раптовому закритті застосунку або втраті зв'язку. Для аутентифікації мають використовуватися безпечні токени (JWT), а збереження персональної інформації в локальній БД (SQLite/Room) має відповідати стандартам шифрування даних на пристрої.

Вимоги до програмного та апаратного забезпечення

Архітектура повинна підтримувати легке додавання нових мовних локалізацій та типів вправ без необхідності радикальної зміни структури коду, що забезпечується використанням патерну MVVM.

1.3 Вибір і обґрунтування архітектурного рішення

Аналіз методів зберігання динамічних структур даних та вибір СУБД

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				18
Змн.	Арк.	№ докум.	Підпис	Дата		

Ефективність мобільного застосунку для навчання критично залежить від швидкості доступу до лінгвістичних даних та надійності збереження прогресу користувача. Архітектурний виклик полягає у необхідності забезпечити сувору цілісність для словникових баз (зв'язки між словами, перекладами та медіафайлами) та гнучкість для збереження персоналізованих навчальних планів.

Метод 1: Патерн EAV (Entity-Attribute-Value) у реляційних базах

Цей підхід передбачає зберігання кожного атрибута слова (транскрипція, приклад, аудіо-лінк) в окремих рядках таблиці знань. Хоча це забезпечує гнучкість, для мобільного пристрою це є критичним антипатерном. Велика кількість операцій JOIN при спробі завантажити один урок з 20 слів створить надмірне навантаження на процесор смартфона, що призведе до затримок інтерфейсу та швидкого розряду акумулятора.

Метод 2: Документо-орієнтовані бази даних (NoSQL) Використання документо-орієнтованих баз дозволяє зберігати урок як єдиний JSON-об'єкт. Це ідеально для швидкості читання, проте створює ризики для цілісності даних. Оскільки навчальний процес вимагає складних зв'язків (одне слово може належати до різних модулів, мати декілька статусів вивчення), відсутність суворих реляційних зв'язків ускладнює контроль за консистентністю даних при їх синхронізації з хмарою.

Метод 3: Гібридний підхід (MySQL з підтримкою JSON) Для розв'язання цього компромісу у проєкті обрано комбіновану модель зберігання. На рівні мобільного клієнта використовується SQLite (через архітектурну надбудову Room для Android або CoreData для iOS). Це стандарт для 121 спеціальності, що забезпечує стабільну роботу в офлайн-режимі. На рівні сервера обрано реляційну СУБД (наприклад, PostgreSQL або MySQL), яка гарантує ACID-транзакційність для профілів користувачів та глобальних словників.

Переваги такого архітектурного рішення для платформи:

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				19
Змн.	Арк.	№ докум.	Підпис	Дата		

- Оптимізація офлайн-доступу: Використання SQLite на пристрої дозволяє учневі продовжувати заняття без інтернету. При відновленні мережі виконується лише дельта-синхронізація (передаються тільки змінені результати).
- Гнучкість через JSON: Конфігурації складних вправ (наприклад, інтерактивні тести з динамічною кількістю варіантів) зберігаються у форматі JSON всередині реляційних таблиць. Це дозволяє додавати нові типи ігрових механік без зміни схеми БД (Zero-migration).
- Продуктивність: Сервер і клієнт отримують повну структуру навчального модуля одним запитом, що мінімізує мережеві затримки.

Таким чином, використання реляційної моделі з інтеграцією JSON-об'єктів забезпечує баланс між швидкістю мобільного інтерфейсу та надійністю збереження освітнього прогресу, що є оптимальним вибором для інженерії програмного забезпечення у сфері EdTech.

Інфраструктура та хмарні сервіси

Для забезпечення високої доступності, синхронізації даних у реальному часі та безпеки мобільного застосунку було обрано хмарну інфраструктуру, орієнтовану на мобільну розробку. Це дозволяє зосередитися на бізнес-логіці навчання, делегуючи підтримку серверної частини надійним провайдерам. Екосистема Firebase (Google Cloud) В якості базової хмарної платформи обрано сервіси Firebase, що є оптимальним рішенням для мобільної інженерії. Платформа забезпечує наступні критичні функції: Firebase Authentication: Використовується для безпечної реєстрації та входу користувачів. Підтримує JWT-токени та вхід через Google/Apple ID, що є стандартом для сучасних освітніх застосунків. Firebase Cloud Messaging (FCM): Інструмент для реалізації push-сповіщень. Це критично для мобільного додатка, оскільки дозволяє нагадувати учневі про час інтервальних повторень, підвищуючи рівень утримання (retention) користувачів. Firebase Crashlytics: Система моніторингу помилок у реальному часі. Дозволяє розробнику оперативно отримувати звіти про збої на різних моделях смартфонів та версіях ОС.

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				20
Змн.	Арк.	№ докум.	Підпис	Дата		

Управління навчальним контентом (Cloudinary та Storage) Для зберігання медіафайлів (аудіо-файли з вимовою слів, ілюстрації до карток) інтегровано хмарний сервіс Cloudinary. Оскільки мобільний пристрій має обмежену пам'ять, зберігання важких медіаданих локально в пакеті додатка є недоцільним.

Cloudinary автоматично оптимізує аудіо та зображення під пропускну здатність мережі користувача, що гарантує швидке завантаження уроків навіть при слабкому 3G/LTE з'єднанні.

Хостинг API та База Даних (Render / Railway) Для реалізації серверної логіки та зберігання глобальної словникової бази використовується платформа Render (або альтернативно Railway). Вона забезпечує розгортання контейнеризованого бекенду, автоматичне налаштування HTTPS-з'єднань та масштабування ресурсів. В якості керованої СУБД обрано PostgreSQL (через сервіс Aiven або Neon), що гарантує ACID-відповідність при синхронізації великих масивів навчальних даних.

Безпека та захист даних (Cloudflare) Для захисту API-ендпоінтів від DDoS-атак та автоматизованих парсерів (які можуть намагатися викрасти словникові бази) використовується Cloudflare. Він виступає захисним проксі-шаром, приховуючи реальну адресу сервера та забезпечуючи шифрування трафіку між смартфоном і хмарою за допомогою SSL/TLS протоколів.

Дистрибуція та CI/CD На відміну від вебсайтів, розгортання мобільного додатка передбачає публікацію у Google Play Store та Apple App Store. Для автоматизації цього процесу (Continuous Integration) використовуються інструменти GitHub Actions або Codemagic. При кожному оновленні коду система автоматично збирає інсталяційний файл (.apk / .aab / .ipa), проводить модульне тестування та готує збірку для тестувальників.

1.4 Вибір стеку технологій та апаратного забезпечення

Вибір засобів клієнтської та серверної розробки

Вибір інструментальних засобів розробки є фундаментальним етапом проєктування, який безпосередньо визначає швидкість створення програмного

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				
Змн.	Арк.	№ докум.	Підпис	Дата		21

продукту, його надійність та можливості подальшого масштабування. Для реалізації мобільного додатка обрано технологічний стек на базі екосистеми JavaScript та її статично типізованого надмножини TypeScript. Використання єдиної мовної парадигми для клієнтської та серверної частин дозволяє суттєво оптимізувати процес розробки завдяки перевикористанню спільних типів даних та бізнес-логіки, що особливо важливо при роботі зі складними освітніми структурами.

Для розробки клієнтської частини обрано фреймворк React Native, який дозволяє створювати кросплатформені мобільні застосунки з нативною продуктивністю. Такий вибір обґрунтований суворим компонентним підходом, де кожен елемент інтерфейсу, як-от картка слова чи блок тестування, інкапсулюється в окремий модуль. На відміну від вебрішень, React Native взаємодіє безпосередньо з нативними компонентами операційних систем iOS та Android, що забезпечує стабільну частоту оновлення екрана на рівні 60 кадрів на секунду та миттєвий відгук на дії користувача.

Серверна частина системи базується на платформі Node.js, яка завдяки своїй асинхронній та подієво-орієнтованій моделі ідеально підходить для обробки інтенсивних запитів на синхронізацію навчальних даних. В якості вебфреймворку використовується Express.js, що надає гнучкі інструменти для побудови REST API та легкої інтеграції механізмів автентифікації на базі JWT-токенів. Таке поєднання технологій дозволяє ефективно керувати великими масивами JSON-даних без блокування потоку виконання, що критично для забезпечення швидкодії серверної частини.

Процес написання коду здійснюється в інтегрованому середовищі Visual Studio Code, яке поєднує в собі легковажність та потужний функціонал для налагодження кросплатформених застосунків. Для моделювання архітектури та візуалізації зв'язків між сутностями обрано інструмент PlantUML, який дозволяє описувати діаграми у текстовому форматі. Це забезпечує можливість зберігання архітектурних схем безпосередньо у

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				
Змн.	Арк.	№ докум.	Підпис	Дата		22

репозиторії Git, дозволяючи відстежувати історію змін проєкту паралельно з вихідним кодом.

Апаратні вимоги до робочої станції розробника зумовлені необхідністю одночасного запуску середовища розробки та емуляторів мобільних пристроїв, що потребує багатоядерного процесора та щонайменше 16 ГБ оперативної пам'яті. Для кінцевого користувача вимоги є більш лояльними: сучасний смартфон під управлінням Android або iOS із обсягом оперативної пам'яті від 2 ГБ та вільним місцем на накопичувачі для кешування навчальних медіафайлів. Стабільне підключення до мережі інтернет є необхідним лише для періодичної синхронізації прогресу, тоді як основна логіка навчання доступна в автономному режимі.

Висновки до першого розділу

У межах першого розділу здійснено комплексне дослідження предметної області та доведено актуальність розробки мобільного застосунку для вивчення іноземних мов. Аналіз популярних на ринку EdTech-рішень продемонстрував необхідність створення продукту, який би гармонійно поєднував у собі методики інтенсивного запам'ятовування з високим рівнем гнучкості при формуванні персоналізованого навчального контенту. Вивчення потреб цільової аудиторії дозволило чітко окреслити сегменти користувачів та формалізувати ключові сценарії взаємодії, що базуються на принципах адаптивного навчання та систематичного повторення лексичного матеріалу.

На основі отриманих результатів було обґрунтовано вибір сучасного технологічного стеку, орієнтованого на кросплатформену мобільну розробку. Доведено доцільність використання фреймворку React Native у поєднанні з мовою TypeScript, що забезпечує стабільну продуктивність інтерфейсу та надійність програмного коду. Для розв'язання проблеми ефективного зберігання даних обрано гібридну модель, яка передбачає використання локальної бази даних SQLite на пристрої для забезпечення офлайн-доступу та реляційної СУБД на серверній частині для глобальної синхронізації прогресу й керування спільними словниковими активами.

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				23
Змн.	Арк.	№ докум.	Підпис	Дата		

Крім того, у розділі було детально сформульовано функціональні вимоги до системи, що базуються на рольовій моделі доступу, та визначено критичні нефункціональні метрики. Особливу увагу приділено параметрам швидкодії мобільного клієнта, безпеці передачі персональних даних через хмарну інфраструктуру та стабільності роботи в умовах обмеженого мережевого з'єднання. Сукупність сформованих вимог, проаналізованих архітектурних підходів та обраних інструментів створює надійний теоретичний і технічний фундамент для подальшого переходу до етапу логічного проєктування та програмної реалізації системи.

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				
Змн.	Арк.	№ докум.	Підпис	Дата		24

РОЗДІЛ 2. ПРОЄКТУВАННЯ ПРОГРАМНОЇ СИСТЕМИ

2.1 Моделювання варіантів використання та поведінки системи

Процес моделювання варіантів використання виступає фундаментом для переходу від теоретичних вимог до практичної побудови архітектури мобільного застосунку. На цьому етапі об'єктно-орієнтованого проєктування визначаються межі програмного комплексу, ідентифікуються ключові дійові особи (актори) та формалізуються типові сценарії їхньої взаємодії з системою. Використання UML-діаграм дозволяє візуалізувати поведінку додатка, перетворюючи розрізнені функціональні запити у структуровану логічну схему, що відображає реальні потреби користувачів освітньої платформи. Детальний опис кожної ролі наведено у таблиці 2.1.

Таблиця – 2.1 Ідентифікація акторів системи

Актор	Опис ролі та зона відповідальності
Гість	Неавторизований користувач. Має доступ до екранів знайомства (Onboarding) та загальної інформації про курси. Може ініціювати процедуру реєстрації або входу в систему.
Учень (Авторизований користувач)	Основний суб'єкт взаємодії. Має повний доступ до персоналізованого навчального середовища: словників, вправ, системи інтервальних повторень та аналітики власного прогресу.
Контент-менеджер	Фахівець, відповідальний за наповнення та актуалізацію освітнього контенту. Працює з глобальними словниковими базами, додає медіафайли (аудіо, зображення) та формує шаблони вправ.
Адміністратор	Технічний керівник платформи. Здійснює нагляд за безпекою, управляє обліковими записами та правами доступу, моніторить навантаження на серверну частину та налаштовує інтеграції з зовнішніми API.

На основі розробленої моделі управління доступом, в архітектурі мобільного застосунку виділено чотирьох незалежних акторів, кожен з яких має чітко визначену зону відповідальності. Гість представляє неавторизованого

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				25
Змн.	Арк.	№ докум.	Підпис	Дата		

відвідувача, чий доступ обмежений ознайомчим екраном та можливістю реєстрації. Авторизований користувач (Учень) є ключовим актором, який взаємодіє з основним функціоналом навчання, словниками та системами тестування. Контент-менеджер відповідає за актуальність навчальних матеріалів та словникових баз, тоді як Адміністратор здійснює повний технічний контроль за життєдіяльністю платформи, керуючи правами доступу та безпекою.

Зважаючи на кросплатформену природу продукту та принципи модульності, усі варіанти використання логічно розподілені на три функціональні простори: публічний інтерфейс, персоналізоване навчальне середовище та зону адміністрування контенту. У публічному просторі реалізуються прецеденти реєстрації, автентифікації та відновлення доступу, а також ознайомлення з документацією та базовими можливостями додатка. Ці сценарії є ідентичними для всіх ролей на початковому етапі взаємодії із застосунком.

Навчальний простір охоплює більшість прецедентів, орієнтованих на активне засвоєння мови. До них належать запуск сесій інтервальних повторень, створення та наповнення персональних словникових модулів, проходження інтерактивних вправ та візуалізація статистики прогресу. Окремо виділяються варіанти використання, пов'язані з налаштуванням профілю, офлайн-завантаженням контенту та експортом накопичених лінгвістичних даних у структуровані формати. Ця зона фокусується на забезпеченні безперервного та адаптивного освітнього треку для кожного учня.

Простір адміністрування та модерації призначений для управління глобальними активами системи. Тут реалізуються сценарії CRUD-операцій над словниковими базами, створення загальних шаблонів уроків, управління медіатекою та обробка запитів користувачів. Адміністративна частина також включає прецеденти моніторингу активності, керування обліковими записами згідно з політиками безпеки та налаштування конфігурацій API для зовнішніх сервісів. Такий розподіл гарантує цілісність системи та дозволяє гнучко масштабувати функціонал без порушення логіки взаємодії між акторами.

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				
Змн.	Арк.	№ докум.	Підпис	Дата		26

На основі виявлених акторів та формалізованих прецедентів побудовано розширену діаграму варіантів використання системи (рис. 2.2).



Рис. 2.2 – Діаграма варіантів використання системи

На основі ідентифікованих акторів та формалізованих прецедентів було розроблено розширену діаграму варіантів використання мобільного застосунку. Візуалізація чітко розмежовує зони відповідальності: актори базового рівня, такі як Гість та Учень, взаємодіють переважно з публічним інтерфейсом та персоналізованим навчальним простором. Водночас службові ролі, зокрема Контент-менеджер та Адміністратор, мають ексклюзивний доступ до інструментів модерації словникових баз та технічного управління платформою.

Розроблена модель слугує надійним фундаментом для подальшого проектування об'єктної структури класів та реляційної бази даних.

Для більш глибокого розуміння логіки функціонування системи та підготовки до розробки алгоритмів було складено специфікації для ключових прецедентів. Специфікація деталізує кожен крок взаємодії актора з мобільним клієнтом, визначає необхідні передумови та описує альтернативні сценарії, що включають обробку помилок валідації чи мережевого з'єднання. Це дозволяє сформулювати чіткий контракт між мобільним застосунком та серверним API, мінімізуючи ризики при програмній реалізації. У таблиці 2.2 наведено специфікацію базового варіанта використання, який забезпечує безпечний вхід до системи та управління користувачькими сесіями.

У таблиці 2.2 наведено специфікацію базового варіанта використання "Реєстрація та Авторизація", яка забезпечує безпечний доступ до робочого простору та управління сесіями користувачів.

Таблиця 2.2 – Специфікація прецеденту "Реєстрація та Авторизація"

Атрибут	Опис
Назва прецеденту	Реєстрація та Авторизація
Актор	Гість
Мета	Отримати персоналізований доступ до навчальних модулів, словників та системи відстеження прогресу.
Передумови	Користувач відкрив застосунок на мобільному пристрої та має активне підключення до мережі.
Основний сценарій (Успішний)	- Гість обирає опцію реєстрації або входу. - Система відображає інтерфейс для вводу облікових даних. - Гість заповнює форму та підтверджує дію. - Клієнтська частина проводить первинну валідацію полів. - Сервер перевіряє дані в БД та генерує безпечний JWT-токен. - Система зберігає сесію та відкриває доступ до головного екрана навчання.
Альтернативний сценарій (Помилки)	4а. Некоректний формат даних: система виводить підказку щодо виправлення помилок.

	5а. Користувач вже існує або дані невірні: сервер повертає помилку автентифікації, застосунок пропонує відновити пароль або змінити дані.
Постумови	Користувач переходить у статус авторизованого «Учня», активується механізм синхронізації його прогресу з хмарию.

Ключовим інноваційним функціоналом розроблюваного застосунку є можливість динамічного формування індивідуальної освітньої траєкторії через інтерактивне управління навчальними модулями. У таблиці 2.3 наведено інженерну специфікацію прецеденту «Формування персоналізованого навчального модуля», який є найбільш складним з точки зору реалізації клієнтської логіки та синхронізації станів даних.

Таблиця 2.3 – Специфікація прецеденту «Формування персоналізованого навчального модуля»

Атрибут	Опис
Назва прецеденту	Формування персоналізованого навчального модуля
Актор	Користувач
Мета	Створити власну структуру навчального блоку (набір слів, вправ та медіафайлів) без необхідності прямого втручання в архітектуру бази даних.
Передумови	Користувач авторизований у застосунку та перейшов у розділ створення контенту.
Основний сценарій (Успішний)	1. Учень ініціює створення нового модуля. 2. Мобільний клієнт завантажує базову JSON-схему конфігурації. 3. Користувач додає лексичні одиниці, використовуючи інтегровані інструменти пошуку та перекладу. 4. Система автоматично підбирає типи вправ (аудіювання, написання, переклад) на основі доданого контенту. 5. Користувач зберігає зміни. 6. Застосунок серіалізує дані у JSON-об'єкт та відправляє PUT запит на Backend API. 7. Сервер валідує структуру та оновлює запис у глобальній базі даних.

Альтернативний сценарій (Помилки)	6а. Відсутність мережі: система перехоплює помилку, сповіщає користувача та кешує зміни локально в SQLite (Room/CoreData) до відновлення зв'язку. 7а. Помилка валідації: сервер відхиляє запит, система пропонує виправити некоректно заповнені поля.
Постумови	Оновлена структура модуля збережена на сервері та доступна для вивчення в офлайн-режимі на мобільному пристрої.

Окрім клієнтського функціоналу, стабільність інформаційної системи підтримується адміністративними процесами контролю якості контенту. Специфікація прецеденту модерації (табл. 2.4) демонструє захисні механізми платформи та логіку зміни станів об'єктів у глобальній базі даних, що дозволяє уникати поширення недостовірної чи некоректної лінгвістичної інформації.

Таблиця 2.4 Специфікація прецеденту «Призупинення доступу до публічного модуля»

Атрибут	Опис
Назва прецеденту	Призупинення доступу до публічного модуля
Актор	Контент-менеджер, Адміністратор
Мета	Обмежити доступ до навчального контенту, що містить грубі помилки, порушує авторські права або політику конфіденційності платформи.
Передумови	Модератор авторизований в адміністративній панелі та ідентифікував проблемний навчальний блок.
Основний сценарій (Успішний)	1. Модератор знаходить модуль через систему фільтрації за ідентифікатором або автором. 2. Обирає дію «Деактивувати». 3. Система вимагає вказати причину блокування. 4. Після підтвердження сервер змінює статус об'єкта в БД з "active" на "suspended". 5. Система автоматично надсилає повідомлення автору з роз'ясненнями та інструкціями щодо виправлення.

Альтернативний сценарій (Помилки)	4а. Спроба блокування системного модуля: система перевіряє права доступу (RBAC) та відхиляє операцію для захищених сутностей.
Постумови	При спробі завантаження модуля мобільні клієнти отримують статус доступу «заборонено», а контент зникає із загального каталогу.

Розроблені специфікації прецедентів визначають архітектурні контракти системи. Вони демонструють, що кожна дія актора супроводжується глибокою серверною логікою: агрегацією масивів статистики, управлінням транзакціями бази даних, валідацією прав доступу, обробкою помилок мережі та синхронізацією станів. Ця деталізація є вхідними даними для наступного етапу – проєктування об'єктно-орієнтованої структури класів модульного моноліта та маршрутів API.

2.2 Проєктування логічної структури бази даних

Обґрунтування вибору СУБД та гібридного підходу

Для забезпечення надійного зберігання інформації та підтримки транзакційної цілісності освітніх даних було обрано реляційну систему управління базами даних. У межах мобільної розробки це реалізується через використання SQLite (з архітектурною надбудовою Room) на боці клієнта та PostgreSQL або MySQL на сервері. Такий підхід гарантує виконання вимог ACID, що є критично важливим для збереження прогресу навчання, словникових запасів та історії успішності користувачів, де будь-яка втрата даних може негативно вплинути на освітній процес.

Зважаючи на специфіку підсистеми персоналізації навчання, де структура навчального модуля може бути максимально динамічною (різна кількість вправ, типів медіаконтенту та варіативність тестів), для зберігання складних конфігурацій використано нативний тип даних JSON. Це дозволяє ідеально поєднати надійність строгих реляційних зв'язків для системних сутностей, таких як профілі користувачів та мовні пари, із гнучкістю документо-орієнтованого підходу для опису логіки інтерактивних завдань. Таке

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				
Змн.	Арк.	№ докум.	Підпис	Дата		31

архітектурне рішення мінімізує необхідність складних міграцій бази даних при додаванні нових типів ігрових механік.

Логічна структура бази даних застосунку охоплює систему взаємопов'язаних таблиць, що забезпечують управління користувачами за рольовою моделлю RBAC, лінгвістичними базами (слова, фрази, переклади), медіатекою та результатами тестувань. Для уникнення надлишковості інформації та підтримки цілісності даних, більшість таблиць реляційної моделі було нормалізовано до третьої нормальної форми (3НФ). Це дозволяє ефективно масштабувати систему, додаючи нові мови або розширюючи статистичні показники без дублювання записів.

Наведений графічний концепт у вигляді ER-діаграми ілюструє ключові сутності системи, їхні атрибути, первинні та зовнішні ключі, а також типи логічних зв'язків. Зокрема, виділяються зв'язки «один-до-багатьох» між користувачами та їхніми персональними словниками, а також складні відношення між лексичними одиницями та мультимедійним контентом. Спроектowana схема формує єдиний інформаційний простір, який забезпечує швидку вибірку даних для мобільного інтерфейсу та надійну синхронізацію між локальним сховищем смартфона і хмарним сервером.

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				32
Змн.	Арк.	№ докум.	Підпис	Дата		

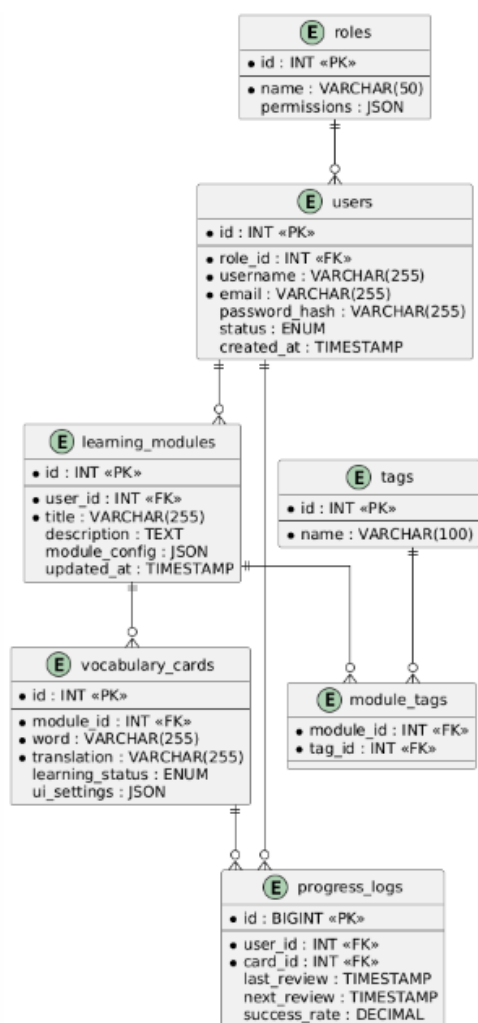


Рис. 2.3 – ER-діаграма реляційної бази даних

Для деталізації зв'язків між сутностями системи та опису архітектури даних було розроблено концептуальну ER-діаграму. Вона відображає ієрархічну структуру мобільного застосунку, де центральне місце займає взаємодія між профілем користувача та його індивідуальним навчальним контентом. Побудова логічних зв'язків базується на принципах цілісності та мінімізації надлишковості, що забезпечує стабільну роботу системи при великих обсягах лінгвістичних даних.

Ключові зв'язки між сутностями системи побудовані за наступними правилами. Між таблицями користувачів та їхніми персональними словниками встановлено зв'язок один-до-багатьох, що дозволяє кожному учневі формувати необмежену кількість тематичних модулів. При цьому видалення облікового запису ініціює каскадне видалення всіх пов'язаних із ним персональних

налаштувань та прогресу, що відповідає політиці конфіденційності та безпеки даних.

Навчальні модулі, у свою чергу, пов'язані із сутністю вправ та карток слів також через зв'язок один-до-багатьох. Кожна одиниця контенту містить унікальне поєднання ідентифікатора модуля та лексичного об'єкта. Для гнучкої класифікації навчального матеріалу реалізовано зв'язок багато-до-багатьох між модулями та категоріями (тегами) через транзитну таблицю, що дозволяє користувачам ефективно фільтрувати контент за рівнем складності або тематикою, наприклад, "Business" або "Travel".

Особливе місце в архітектурі займає модуль відстеження прогресу, який пов'язує результати тестувань із конкретними словами через таблицю статистики вивчення. Це дозволяє зберігати історію взаємодії користувача з кожною лексичною одиницею, фіксувати кількість помилок та дату наступного повторення згідно з алгоритмом інтервальних повторень. Така структура гарантує історичну достовірність прогресу навіть при зміні загальних параметрів курсу.

На основі логічної моделі було спроектовано фізичну структуру бази даних. У таблиці 2.5 наведено специфікацію сутності, що відповідає за зберігання облікових записів та налаштувань доступу.

Таблиця 2.5 – Структура таблиці users

Назва поля	Тип даних	Обмеження	Опис
id	INT	PK, AUTO_INCREMENT	Унікальний ідентифікатор користувача.
username	VARCHAR(255)	UNIQUE, NOT NULL	Публічне ім'я (нікнейм) учня в системі.
email	VARCHAR(255)	UNIQUE, NOT NULL	Електронна пошта для авторизації та сповіщень.

password_hash	VARCHAR(255)	NULL	Хешований пароль; може бути порожнім при вході через Google/Apple.
role	ENUM	NOT NULL, DEFAULT 'user'	Роль у системі ('user', 'moderator', 'admin').
language_level	ENUM	NOT NULL, DEFAULT 'A1'	Поточний рівень володіння іноземною мовою.
status	ENUM	NOT NULL, DEFAULT 'active'	Стан акаунта ('active', 'inactive', 'banned').
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Дата створення профілю.

Центральною сутністю модуля управління контентом є навчальний модуль. Таблиця learning_modules містить загальні метадані освітнього блоку, параметри доступу та конфігураційні ключі для персоналізації процесу навчання. Фізична структура цієї таблиці, наведена нижче, забезпечує гнучке керування видимістю матеріалів та дозволяє інтегрувати модулі у загальну освітню екосистему застосунку.

Таблиця 2.6 – Структура таблиці learning_modules

Назва поля	Тип даних	Обмеження	Опис
id	INT	PK, AUTO_INCREMENT	Унікальний ідентифікатор навчального модуля.
user_id	INT	FK, NOT NULL	Посилання на власника (автора) модуля.
slug	VARCHAR(255)	UNIQUE, NOT NULL	Унікальний ідентифікатор для поширення модуля або офлайн-доступу.

title	VARCHAR(255)	NOT NULL	Назва навчального курсу або набору слів.
status	ENUM	NOT NULL, DEFAULT 'private'	Статус доступу ('private', 'public', 'shared').
module_config	JSON	NULL	Глобальні налаштування черговості вправ та логіки навчання у форматі JSON.
sync_key	VARCHAR(255)	NULL	Ключ для верифікації цілісності даних при хмарній синхронізації.

Для зберігання безпосереднього наповнення, яке користувач формує через інтерфейс редактора карток, використовується таблиця vocabulary_cards. Кожен запис у цій таблиці представляє ізольовану лексичну одиницю з набором асоціативних зв'язків, що належить конкретному навчальному модулю. Це дозволяє реалізувати атомарний підхід до управління знаннями, де кожне слово має власний набір візуальних та технічних параметрів.

Таблиця 2.7 – Структура таблиці vocabulary_cards

Назва поля	Тип даних	Обмеження	Опис
id	INT	PK, AUTO_INCREMENT	Унікальний ідентифікатор картки.
module_id	INT	FK, NOT NULL	Посилання на модуль, до якого входить слово.
word_type	VARCHAR(50)	NOT NULL	Категорія (наприклад, 'noun', 'verb', 'phrase').

content_data	JSON	NOT NULL	Параметри лінгвістичного наповнення (переклад, приклади, транскрипція).
ui_settings	JSON	NOT NULL	Конфігурація відображення картки (стилі, посилання на медіафайли).

Для забезпечення безпеки, аудиту змін у лінгвістичних базах та контролю за діями модераторів реалізовано підсистему журналювання транзакцій. Структура таблиці audit_logs дозволяє відстежувати історію модифікації критичних даних, що важливо для відновлення станів системи у разі помилок або несанкціонованого втручання.

Таблиця 2.8 – Структура таблиці audit_logs

Назва поля	Тип даних	Обмеження	Опис
id	BIGINT	PK, AUTO_INCREMENT	Унікальний ідентифікатор запису логу.
user_id	INT	FK, NOT NULL	Ідентифікатор особи, що виконала зміну.
action	VARCHAR(100)	NOT NULL	Тип операції (наприклад, 'EDIT_WORD', 'DELETE_MODULE').
target_table	VARCHAR(50)	NOT NULL	Сутність, яка зазнала змін.
record_id	INT	NOT NULL	Ідентифікатор конкретного об'єкта в базі.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Точний час виконання операції.

Опис архітектури JSON-схеми аналітичного звіту

Ключовим архітектурним рішенням платформи є спосіб збереження структури навчального контенту через JSON-схеми. Замість створення надмірної кількості таблиць для кожного нового типу вправи чи формату медіаданих, поля module_config та content_data використовують нативний формат JSON. Це

забезпечує гнучкість, характерну для NoSQL-рішень, зберігаючи при цьому всі переваги реляційної цілісності основної бази даних. Такий підхід дозволяє розробнику додавати нові методики навчання (наприклад, розпізнавання мовлення чи асоціативні ігри) без необхідності складних маніпуляцій зі схемою бази даних, що значно прискорює цикл розробки та оновлення мобільного застосунку.

Дані у системі зберігаються у формі впорядкованого масиву об'єктів, де кожен елемент відповідає за конкретний функціональний блок або навчальний віджет у межах мобільного інтерфейсу. Такий підхід надає максимальну гнучкість при масштабуванні платформи: впровадження нових типів ігрових механік чи вправ не потребує виконання складних міграцій у реляційній структурі бази даних. Замість цього розробнику достатньо оновити логіку обробки JSON-схеми на боці клієнтського застосунку.

Кожен стандартний вузол навчальної схеми має суворо визначену структуру та містить набір ключових параметрів. Унікальний ідентифікатор блоку забезпечує коректну роботу алгоритмів рендерингу та дозволяє динамічно керувати станом елементів на екрані смартфона. Параметр типу константи визначає, який саме освітній компонент має бути завантажений фабрикою рендерингу — це може бути картка для запам'ятовування, інтерактивний тест або аудіо-модуль. Вкладені об'єкти містять параметри лінгвістичного контенту (вибірки слів, переклади, медіа-ресурси) та візуальні конфігурації, що визначають стиль відображення конкретного завдання.

Застосована архітектура гарантує високу продуктивність системи. Під час завантаження нового навчального модуля сервер виконує лише один атомарний запит до бази даних, отримує повну JSON-конфігурацію та миттєво передає її на мобільний пристрій. Клієнтська частина на базі React Native відмальовує структуру інтерфейсу та паралельно ініціює асинхронні запити на отримання медіафайлів із CDN, що забезпечує безперервність користувацького досвіду без затримок та очікувань.

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				38
Змн.	Арк.	№ докум.	Підпис	Дата		

2.3 Проектування компонентної структури та взаємодії модулів

Діаграма розгортання призначена для моделювання фізичної та інфраструктурної топології системи, відображаючи конфігурацію серверних потужностей та хмарних сервісів. Вона дозволяє оцінити розподіл навантаження між окремими компонентами платформи та деталізувати протоколи їхньої взаємодії. Оскільки мобільний застосунок проєктується як сучасне хмарне рішення, його архітектура базується на принципах розподіленої інфраструктури, що забезпечує відмовостійкість та швидку доставку контенту.

На представленій діаграмі розгортання (рис. 2.4) чітко виділено ключові вузли системи. Мобільний пристрій виступає основним клієнтом, який взаємодіє з хмарою через захищений шлюз. Обробка бізнес-логіки відбувається на серверній платформі, тоді як збереження структурованих даних делеговано керованим сервісам баз даних. Окремі SaaS-рішення, такі як Cloudinary для медіафайлів та Cloudflare для захисту трафіку, інтегровані в єдину мережу, що дозволяє оптимізувати маршрутизацію запитів та гарантувати стабільну роботу додатка навіть при високому навантаженні.

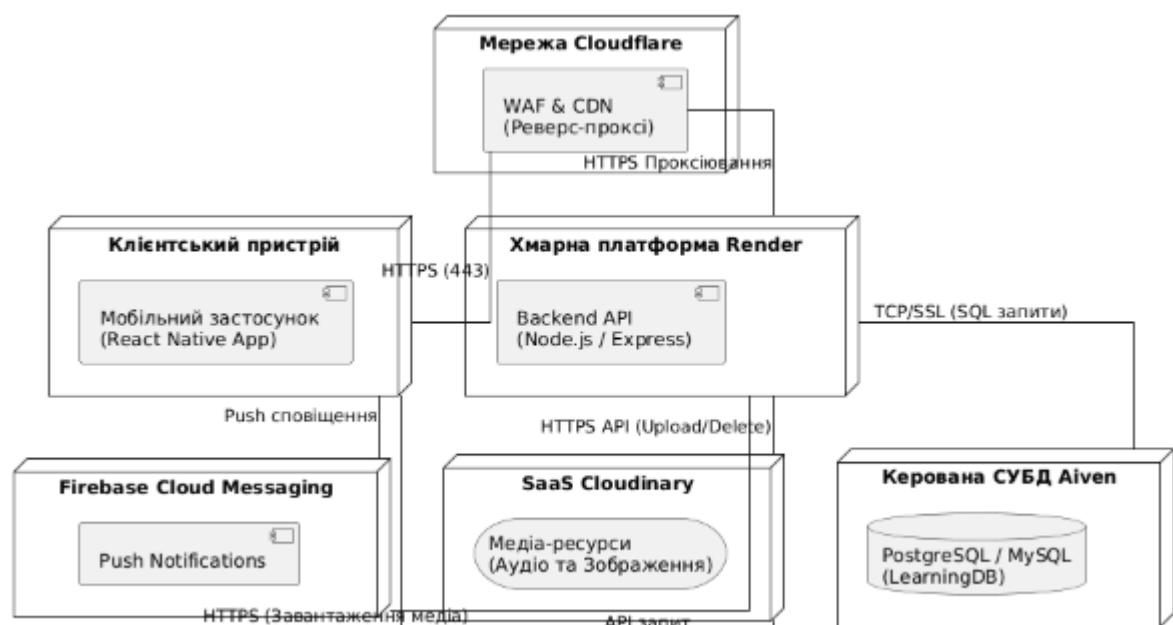


Рис. 2.4 – Діаграма розгортання системи

Архітектурна модель платформи базується на розподіленій хмарній інфраструктурі, що забезпечує стабільну роботу мобільного застосунку та надійну синхронізацію навчальних даних. Основним вузлом взаємодії є

мобільний пристрій користувача, на якому виконується артефакт клієнтського застосунку, розроблений на базі React Native. Цей компонент відповідає за рендеринг інтерфейсу навчання, управління локальним станом словників у SQLite та забезпечення миттєвої реакції на дії учня незалежно від якості мережевого з'єднання.

Безпеку та маршрутизацію трафіку забезпечує мережа Cloudflare, яка виступає захисним бар'єром між клієнтом та серверною частиною. Цей вузол виконує функції реверс-проксі, фільтруючи автоматизований зловмисний трафік та забезпечуючи шифрування даних за допомогою SSL-протоколів. Завдяки Cloudflare запити користувача безпечно спрямовуються до відповідних хмарних ресурсів, що гарантує високу доступність системи та захист лінгвістичних баз даних від несанкціонованого копіювання.

Серверна частина системи розгорнута на платформі Render, де функціонує основна бізнес-логіка додатка у середовищі Node.js. Цей вузол обробляє запити до API, керує процесами автентифікації, реалізує алгоритми інтервальних повторень та забезпечує транзакційну взаємодію з базою даних. Хмарна інфраструктура дозволяє серверу автоматично масштабуватися при зростанні активності користувачів, підтримуючи стабільну швидкість обробки даних при синхронізації прогресу навчання.

Зберігання структурованої інформації та словникових активів реалізовано за допомогою керованої бази даних на платформі Aiven. Вузловий сервер бази даних працює в ізольованому середовищі, що унеможливорює прямий зовнішній доступ та гарантує цілісність освітнього контенту. Для роботи з мультимедійними ресурсами, такими як аудіо-файли з вимовою та ілюстрації до карток, інтегровано сервіс Cloudinary. Він забезпечує динамічну оптимізацію контенту під параметри конкретного мобільного пристрою, що критично важливо для збереження трафіку та швидкодії застосунку.

Архітектурна взаємодія компонентів системи

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				40
Змн.	Арк.	№ докум.	Підпис	Дата		

Сценарій архітектурної взаємодії у системі ініціюється мобільним клієнтом. Коли користувач запускає застосунок, первинні запити проходять через захисні фільтри Cloudflare, які забезпечують безпечне з'єднання та перевірку цілісності сесії. Мобільний клієнт, побудований на базі React Native, звертається до хмарного бекенду за маршрутом /api/* для отримання актуальних навчальних планів або синхронізації прогресу вивчення мови. Cloudflare ідентифікує ці API-запити та безпечно проксіює їх безпосередньо на сервер Render, де розгорнута основна бізнес-логіка.

Node.js-сервер приймає вхідний запит і маршрутизує його до відповідного функціонального модуля всередині структури модульного моноліта — наприклад, до домену «Навчання» або «Словник». Виконуючи необхідні операції, сервер ініціює оптимізований SQL-запит до керованої бази даних на платформі Aiven. Отримавши результат (наприклад, перелік слів для інтервального повторення), сервер формує структуровану JSON-відповідь, яка повертається мобільному клієнту для миттєвого оновлення локального стану та візуалізації на екрані смартфона.

Окремий архітектурний потік виділено для роботи з мультимедійним контентом. Під час додавання нових лінгвістичних матеріалів модератором, бекенд передає аудіо- та графічні файли до Cloudinary через захищене API, зберігаючи у базі даних лише оптимізовані посилання. У процесі навчання мобільні пристрої завантажують ці ресурси (вимову слів, ілюстрації до карток) напряму з глобальної мережі CDN Cloudinary. Це дозволяє повністю оминати основний сервер платформи при завантаженні «важкого» контенту, що значно знижує мережеве навантаження та пришвидшує роботу інтерфейсу.

Така розподілена хмарна архітектура гарантує високу відмовостійкість, миттєву доставку освітніх матеріалів та надійний захист персональних даних учнів. Програмна реалізація системи базується на принципах модульності та суворого розділення відповідальності (Separation of Concerns). Для проєктування структури коду застосовано методології об'єктно-орієнтованого

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				
Змн.	Арк.	№ докум.	Підпис	Дата		41

програмування та сучасні архітектурні патерни, що забезпечують слабку зв'язність компонентів.

Об'єктно-орієнтована доменна модель системи

Доменна модель описує фундаментальні сутності освітньої сфери (LearningModule, VocabularyCard, ProgressLog, User) та логічні зв'язки між ними, абстрагуючись від особливостей технічної реалізації. Вона формує ядро бізнес-логіки, визначаючи, як знання структурується та перетворюється на інтерактивний навчальний досвід. На рис. 2.5 представлено діаграму класів предметної області, яка відображає ієрархію об'єктів мобільного застосунку та їхню взаємодію в межах освітнього процесу.

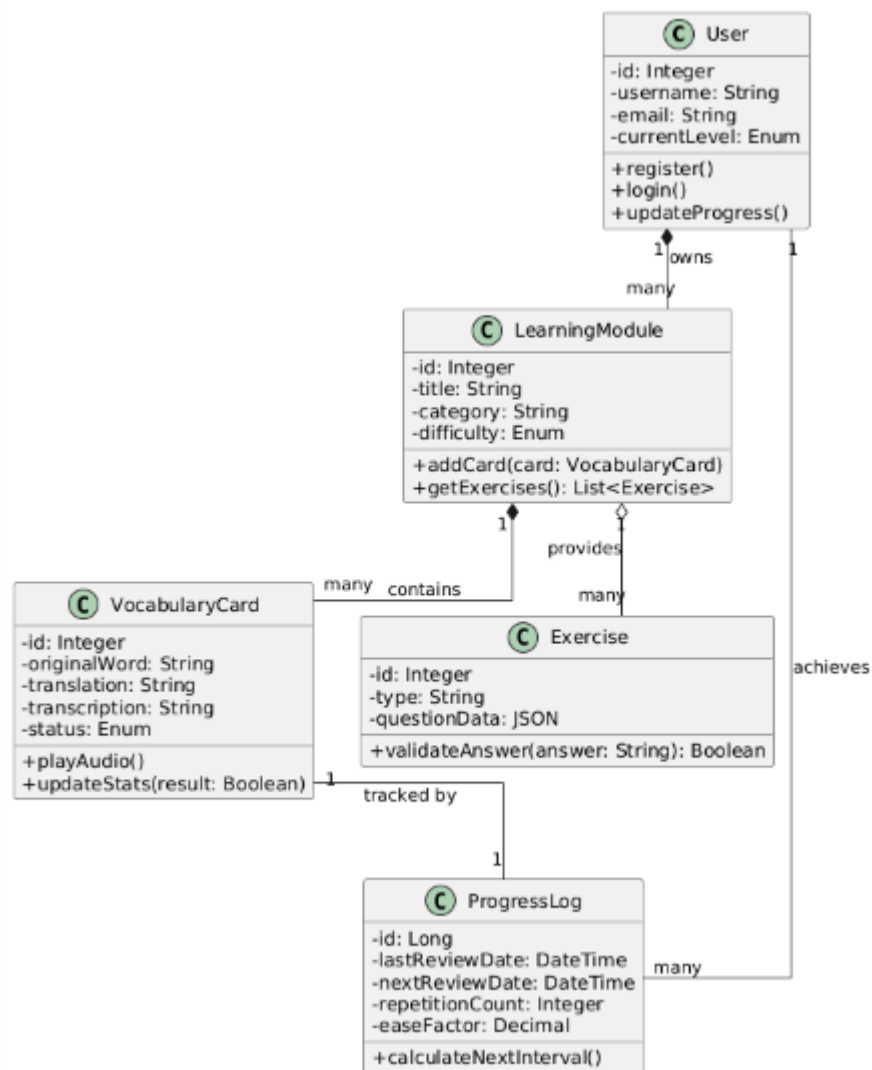


Рис. 2.5 – Діаграма класів предметної області

Центральною сутністю предметної області є клас User, який представляє обліковий запис учня. Він перебуває у відношенні композиції до

класів LearningModule, Media та ProgressLog, оскільки ці об'єкти безпосередньо створюються користувачем та існують у межах його профілю. Клас LearningModule виступає агрегатором навчального контенту, перебуваючи у суворій композиції з класом VocabularyCard. Ключовими атрибутами картинок є content_data та ui_settings типу JSON, де зберігається лінгвістичне наповнення та візуальна конфігурація. Додатково модулі взаємодіють із глобальними категоріями та медіатекою, формуючи запити на отримання необхідних для навчання активів.

Патерни проєктування та архітектура реалізації

Для забезпечення масштабованості коду в межах архітектури та уникнення його дублювання, розробка спирається на використання сталих інженерних патернів. Діаграма класів реалізації ілюструє проходження потоку даних від мобільного застосунку до бази даних, відображаючи чіткий поділ відповідальності між компонентами системи.

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				43
Змн.	Арк.	№ докум.	Підпис	Дата		

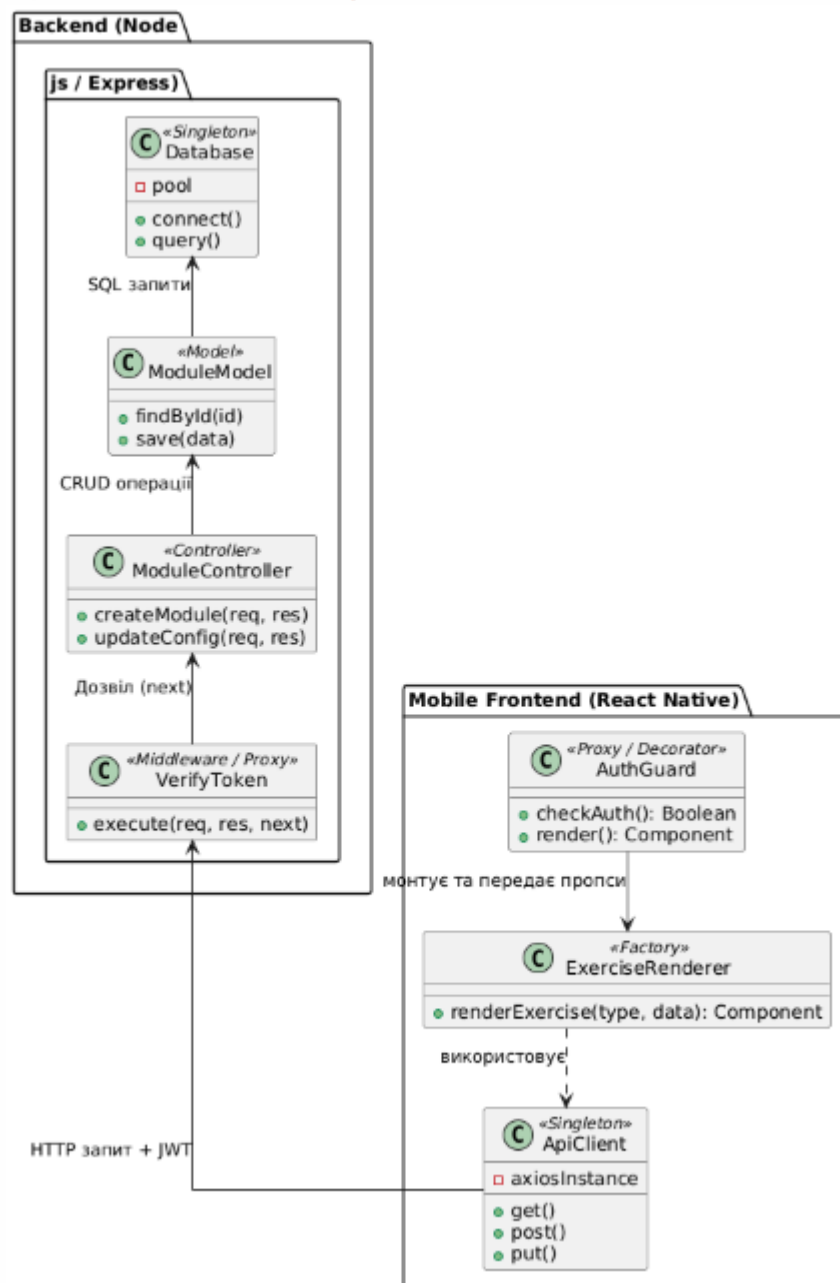


Рис. 2.6 – Діаграма класів реалізації з використанням патернів

У системі реалізовано такі архітектурні патерни:

MVVM (Model-View-ViewModel): На сервері використовується класичний MVC, де контролер (наприклад, ModuleController) керує бізнес-логікою, а модель (ModuleModel) забезпечує взаємодію з БД. На клієнті (React Native) застосовується MVVM, де роль View виконують компоненти, а логіка стану винесена у ViewModel, що забезпечує реактивне оновлення інтерфейсу при зміні даних.

Factory (Фабрика): Використовується на фронтенді (клас ExerciseRenderer) для динамічного створення різних типів вправ. Залежно від переданого типу (наприклад, ChoiceTask чи WritingTask), фабрика інстанціює відповідний нативний компонент, уникаючи жорсткого програмування кожного уроку в макеті.

Singleton (Одинак): Застосовується для глобальних сервісів, які вимагають єдиного екземпляра в пам'яті. У мобільному застосунку це ApiClient(налаштований екземпляр axios із JWT-перехоплювачами), а на сервері — клас Database, що керує пулом з'єднань, запобігаючи перевитраті ресурсів при інтенсивних запитах.

Proxy / Decorator (Замісник): Реалізований через компоненти вищого порядку у мобільному клієнті (наприклад, AuthGuard для обмеження доступу) та як проміжне програмне забезпечення (VerifyToken) в Express.js для валідації токенів перед доступом до конфіденційних методів API.

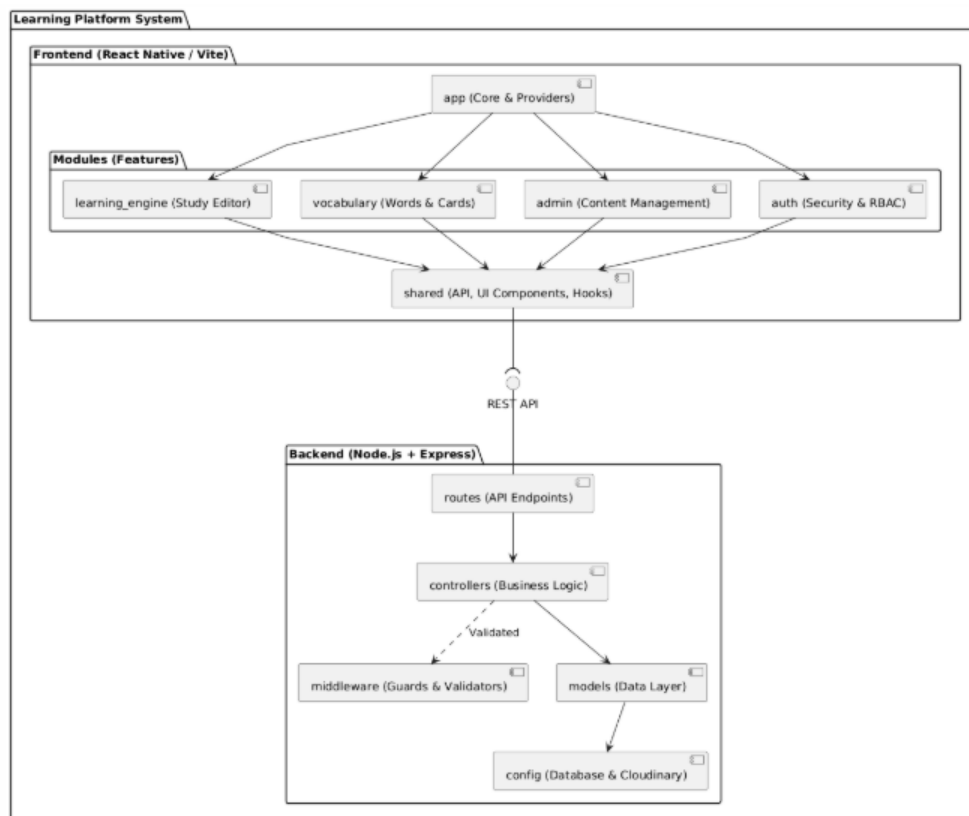


Рис. 2.7 – Діаграма компонентів програмного комплексу

Серверна частина мобільного застосунку, розроблена на базі Node.js та Express, реалізована за принципами модульного моноліта. Структурна

організація коду передбачає чіткий поділ на рівні: папка routes відповідає за декларацію кінцевих точок API, спрямовуючи вхідний трафік до відповідних контролерів (controllers). Безпека системи забезпечується шаром middleware, який виконує роль захисного екрана, здійснюючи перевірку JWT-токенів та валідацію ролей згідно з моделлю RBAC. Усі зовнішні інтеграції, включаючи конфігурацію медіасервісу Cloudinary та налаштування підключення до бази даних Aiven, винесені в ізольований модуль config.

Клієнтська частина, побудована на React Native, спроектована за методологією Feature-Sliced Design, що забезпечує сувору модульну архітектуру та полегшує підтримку коду. Програмний комплекс розділено на три фундаментальні логічні блоки:

app: Ядро мобільного застосунку, яке містить глобальні провайдери стану (авторизація, локалізація), конфігурацію навігації та ініціалізацію локальної бази даних SQLite.

modules (features): Незалежні предметні модулі, що інкапсулюють ключову бізнес-логіку. Найскладнішим є модуль learning_engine, який відповідає за механіку інтервальних повторень, фабрику рендерингу вправ та обробку результатів тестування. Також сюди входять модулі admin для модерації контенту, vocabulary для управління словником та auth для автентифікації користувачів.

shared: Спільний простір, що об'єднує універсальні UI-компоненти (кнопки, інпути, анімації), ізольовані типи карток знань, утиліти для обробки тексту та кастомні хуки (наприклад, useOfflineSync для фонові синхронізації прогресу).

Така декомпозиція дозволяє незалежно тестувати та масштабувати окремі частини освітньої платформи без ризику порушення цілісності загальної логіки. Повна структура каталогів монорепозіторію відображає фізичну організацію проєкту, де кожен компонент має чітко визначене місце.

Проектування взаємодії компонентів та програмних інтерфейсів

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				46
Змн.	Арк.	№ докум.	Підпис	Дата		

Для деталізації процесів обміну даними між мобільним клієнтом та хмарним бекендом було розроблено діаграми взаємодії. Вони візуалізують життєвий цикл обробки HTTP-запитів у межах REST API архітектури, передачу управління між об'єктами та безпосередні звернення до бази даних. Це дозволяє проаналізувати ефективність маршрутизації та виявити можливі вузькі місця у логіці обробки інформації ще на етапі проєктування.

Діаграми послідовності

Діаграми послідовності відображають хронологічний порядок виклику методів та обміну повідомленнями між екземплярами класів під час виконання конкретних прецедентів. На рисунку 2.8 зображено діаграму послідовності процесу ініціалізації нового навчального модуля, що є базовим функціоналом для формування індивідуального освітнього треку.

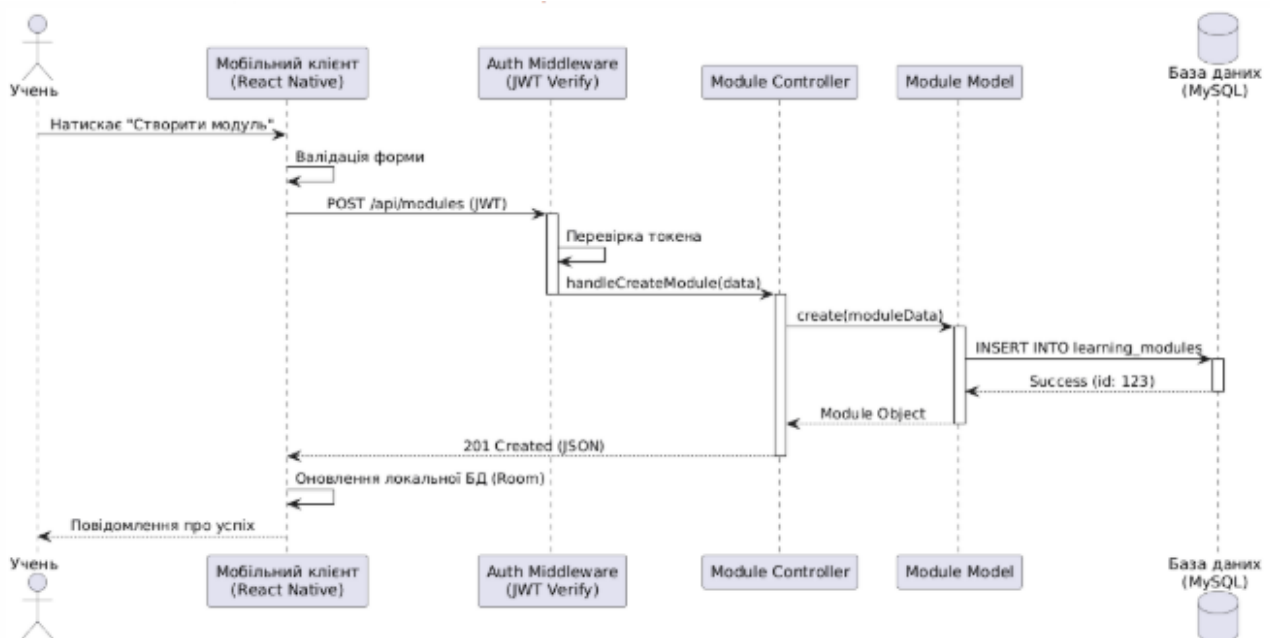


Рис. 2.8 – Діаграма послідовності процесу ініціалізації нового навчального модуля

Взаємодія між об'єктами під час ініціалізації нового навчального модуля або освітнього проєкту відбувається у чіткій хронологічній послідовності. Спочатку актор (Учень) взаємодіє з мобільним інтерфейсом CreateModuleModal, заповнюючи форму базовими параметрами: назвою курсу, мовною парою та налаштуваннями приватності. Компонент інтерфейсу виконує

локальну синхронну валідацію полів, перевіряючи коректність заповнення та наявність обов'язкових атрибутів.

Після підтвердження дії користувачем, мобільний клієнт формує асинхронний HTTP POST запит на маршрут `/api/modules/create` і передає JSON-пакет із конфігураційними даними. Запит проходить через шари безпеки, обробляється маршрутизатором та делегується до `ModuleController`. На цьому етапі контролер звертається до моделі `TemplateModel` для отримання базового лінгвістичного контенту або стартової структури вправ, якщо було обрано готовий шаблон навчання.

На рисунку 2.9 представлена діаграму послідовності створення звернення (тікету) до контент-менеджерів або служби підтримки. Цей процес є критично важливим для забезпечення високої якості освітнього контенту, оскільки дозволяє користувачам повідомляти про помилки в перекладах, некоректну вимову або пропонувати додавання нових лексичних тем.

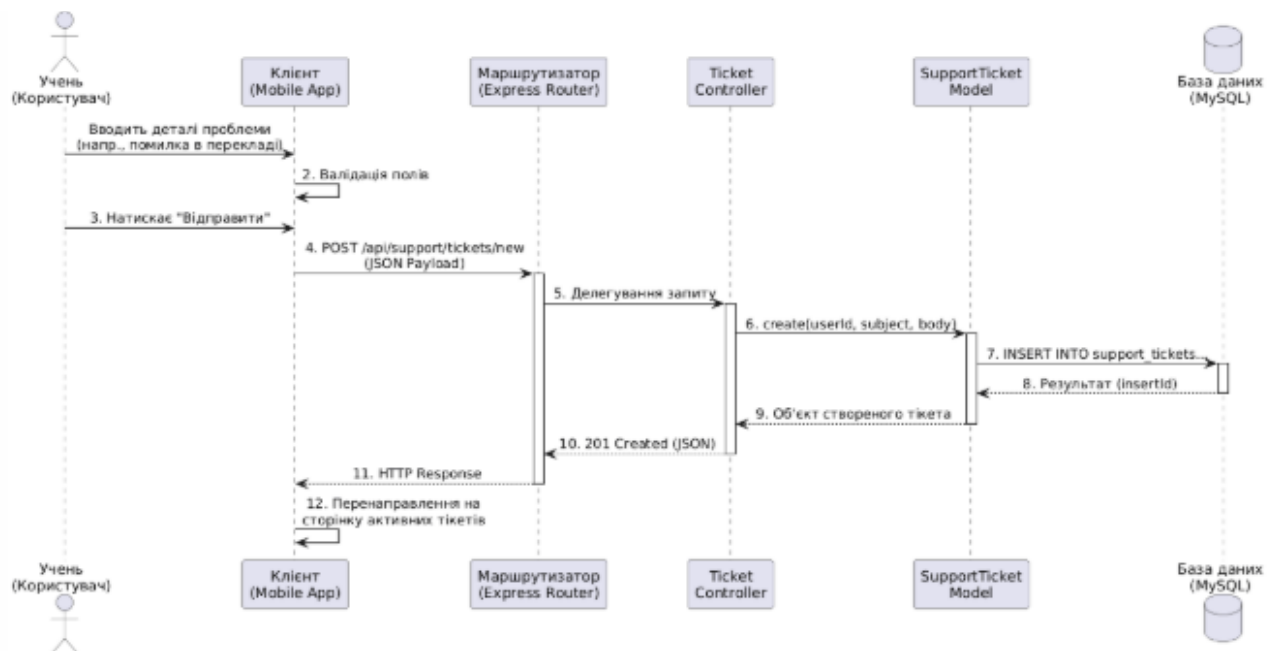


Рис. 2.9 – Діаграма послідовності "Створення звернення до служби підтримки"

Логіка обміну даними при формуванні звернення до підтримки передбачає кілька етапів. Актор (Учень) вводить деталі проблеми у відповідному розділі застосунку, після чого інтерфейс здійснює локальну перевірку полів.

Користувач підтверджує відправку форми, що ініціює HTTP POST запит до TicketController. Контролер передає дані моделі SupportTicketModel, яка виконує запис у базу даних. Після успішного завершення транзакції сервер повертає статус 201 Created, а мобільний застосунок перенаправляє користувача до переліку активних запитів для відстеження статусу модерації.

Отримавши контент шаблону, контролер викликає метод create() моделі ModuleModel, передаючи ідентифікатор користувача та параметри майбутнього модуля. Після успішного виконання операції в базі даних, сервер формує відповідь і повертає об'єкт нового навчального блоку клієнту. Мобільний застосунок завершує транзакцію, оновлює локальний кеш і перенаправляє учня безпосередньо до робочого простору створеного модуля для початку навчання.

На рисунку 2.10 ілюструється складна бізнес-транзакція — процес модерації та призупинення публічного доступу до навчального контенту адміністратором платформи. Цей сценарій містить розгалуження та умовні виклики, що забезпечують захист екосистеми застосунку від неякісного або шкідливого наповнення.

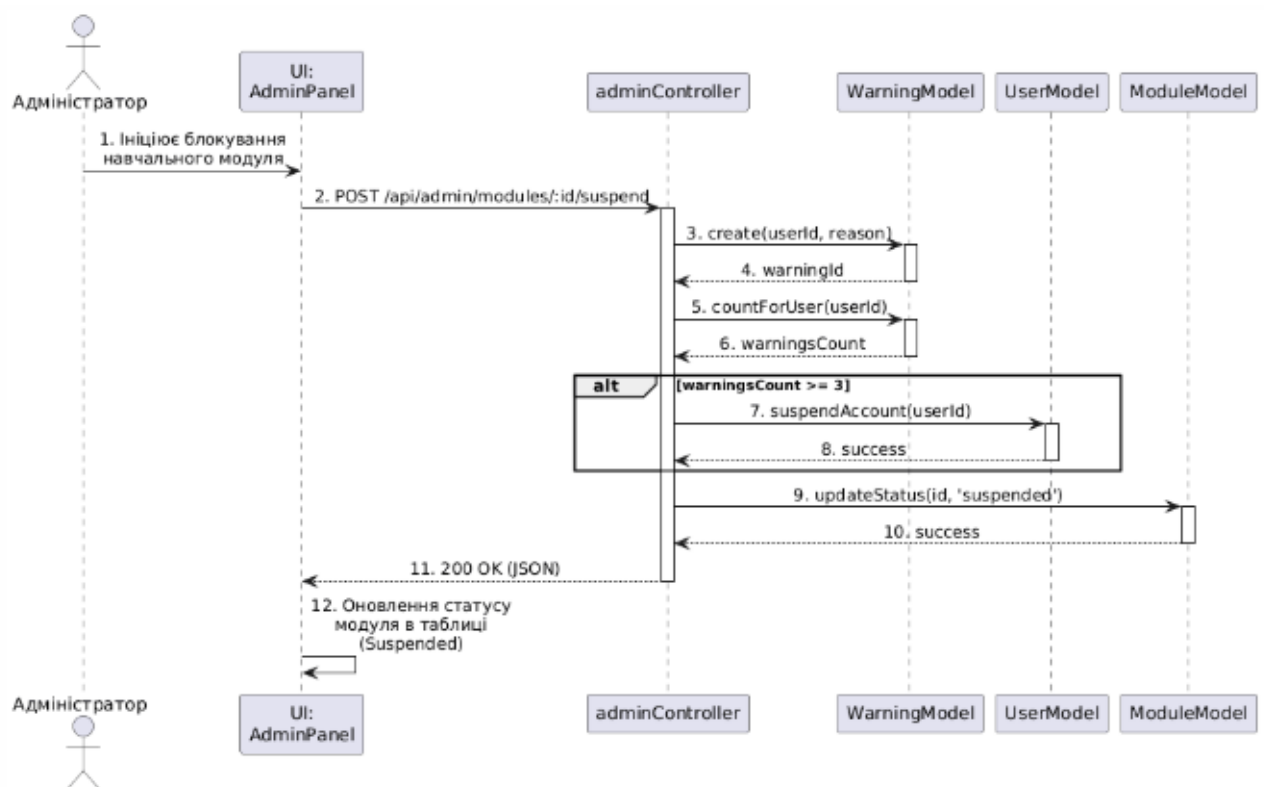


Рис. 2.10 – Діаграма послідовності "Призупинення аналітичного дашборда"

Хронологія адміністративної модерації розпочинається з того, що Адміністратор через панель керування ініціює блокування модуля-порушника. Система відправляє захищений HTTP POST запит до adminController, який першочергово звертається до моделі WarningModel для фіксації офіційного попередження в профілі користувача. Це дозволяє накопичувати історію порушень для подальшого прийняття системних рішень.

Після фіксації попередження контролер виконує повторний запит до моделі для отримання загальної кількості активних страв (попереджень) даного користувача. На цьому етапі реалізовано умовне розгалуження: якщо кількість порушень досягає критичної межі (три або більше), система автоматично звертається до UserModel для призупинення дії всього облікового запису. Це запобігає масовому поширенню неякісного контенту від одного джерела.

Незалежно від результату перевірки ліміту попереджень, контролер надсилає команду до ModuleModel для негайної зміни статусу конкретного навчального ресурсу на "Suspended". Після успішного оновлення записів у базі даних, сервер повертає статус 200 OK мобільному клієнту. На завершальному етапі інтерфейс адміністратора синхронізується з хмарою, візуально позначаючи навчальний блок як заблокований.

Діаграма кооперації (комунікації)

Для глибшого розуміння структурних зв'язків між об'єктами під час виконання складних транзакцій використовується діаграма кооперації (комунікації). Вона трансліює ту ж логіку повідомлень, що й діаграма послідовності, проте зміщує акцент із хронології на просторову архітектуру та топологію зв'язків між програмними модулями..

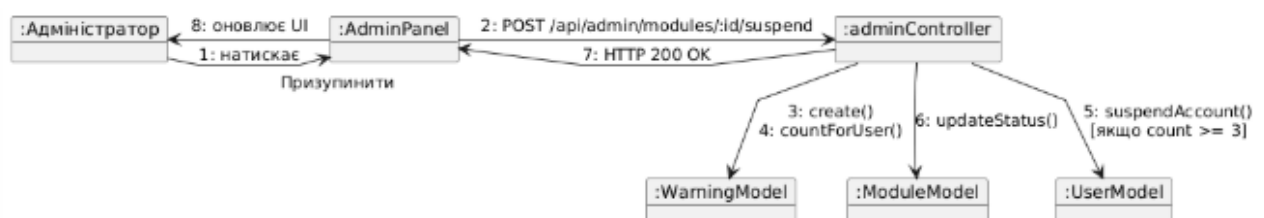


Рис. 2.11 – Діаграма кооперації "Призупинення аналітичного дашборда"

Діаграма кооперації, представлена на рисунку 2.11, демонструє радіальну топологію управління, де центральним вузлом виступає `adminController`. Він виконує функцію фасадного інтерфейсу, що ізолює мобільний застосунок від прямої взаємодії з шаром даних. Така архітектура дозволяє контролеру ефективно координувати роботу трьох незалежних моделей — `WarningModel`, `UserModel` та `ModuleModel`. Це забезпечує суворе дотримання принципу єдиної відповідальності (Single Responsibility Principle) та гарантує транзакційну цілісність під час модерації навчального контенту.

2.4 Розробка основних алгоритмів роботи платформи

Для детального розуміння внутрішньої логіки застосунку було розроблено алгоритми ключових бізнес-процесів, візуалізовані за допомогою діаграм активності. Ці моделі відображають динаміку поведінки системи, ілюструючи паралельні обчислення, логічні розгалуження та чіткий розподіл зон відповідальності між мобільним клієнтом та серверною частиною за допомогою функціональних доріжок.

Алгоритм процесу "Формування навчального модуля"

Даний алгоритм описує один із фундаментальних процесів платформи — ініціалізацію та налаштування нового освітнього блоку на базі обраного шаблону структури. Процес розпочинається з ініціації запиту користувачем, після чого система проходить через етапи валідації, вибору конфігурації та синхронізації з хмарним сховищем. Діаграма активності дозволяє простежити шлях даних від моменту введення інформації учнем до остаточного збереження в базі даних та підготовки контенту для офлайн-використання.

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				
Змн.	Арк.	№ докум.	Підпис	Дата		51



Рис. 2.12 – Діаграма активності " Створення навчального модуля"

Покроковий опис алгоритму:

Ініціалізація: Процес розпочинається з дії користувача (Учня), який натискає кнопку створення нового модуля. Система миттєво звертається до серверного API для завантаження переліку доступних освітніх шаблонів та мовних конфігурацій із таблиці templates.

Введення даних: Учень обирає відповідний шаблон, вводить назву модуля та унікальний ідентифікатор (slug), який використовуватиметься для формування посилань та синхронізації контенту.

Паралельна валідація: На стороні Backend-сервера одночасно запускаються два процеси: перевірка синтаксичної коректності введених даних

(формат тексту, довжина назви) та верифікація лімітів користувача (кількість активних модулів згідно з поточним рівнем доступу).

Перевірка унікальності: У разі успішного проходження попередніх перевірок, система виконує SQL-запит до таблиці learning_modules. Це необхідно для підтвердження того, що обраний ідентифікатор (slug) є вільним у системі, що запобігає конфліктам адрес при поширенні контенту.

Завершення та синхронізація: При успішному виконанні всіх умов у базі даних створюється новий запис. У цей момент копіюється дефолтна JSON-конфігурація вправ із обраного шаблону, після чого мобільний застосунок автоматично перенаправляє користувача до робочого простору для наповнення модуля словами.

Алгоритм процесу "Редагування аналітичного контенту"

Цей процес демонструє роботу з візуальним конструктором звітів, де основне навантаження лягає на клієнтську частину, а звернення до сервера мінімізовані для забезпечення плавності інтерфейсу.

		Сокульський О.І.			ДУ «Житомирська політехніка».26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				53
Змн.	Арк.	№ докум.	Підпис	Дата		

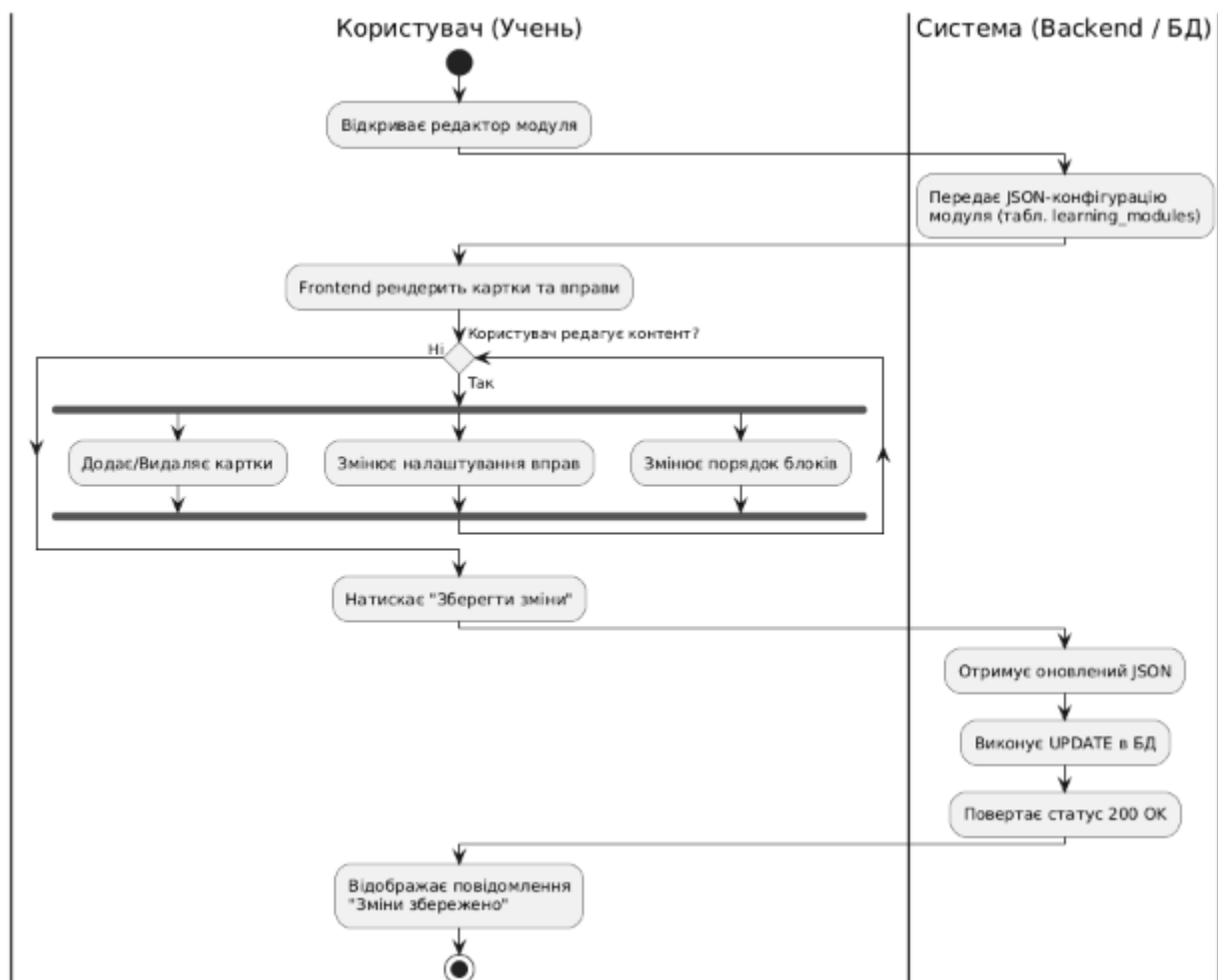


Рис. 2.13 – Діаграма активності "Редагування навчального контенту"

Покроковий опис алгоритму:

1. **Завантаження стану:** Під час ініціалізації редактора серверна частина передає мобільному клієнту актуальний масив JSON-конфігурацій модулів із бази даних. Це дозволяє застосунку отримати повну структуру уроку для подальшої локальної маніпуляції.
2. **Рендеринг:** Клієнтська частина на базі React Native виконує рендеринг інтерактивних компонентів (карток, вправ, медіаблоків) відповідно до отриманої схеми, підготовлюючи робочу область для користувача.
3. **Взаємодія та локальні зміни:** Користувач здійснює маніпуляції з контентом у локальній пам'яті пристрою. Це включає додавання або видалення лексичних одиниць, зміну параметрів вправ та перевпорядкування блоків. Завдяки використанню локального стану (State), усі зміни відображаються миттєво.

4. **Синхронізація:** Після завершення редагування та натискання кнопки збереження, мобільний застосунок формує фінальний оновлений JSON-пакет, який відображає нову структуру всього модуля.
5. **Фізичне збереження:** Серверна частина приймає оновлені дані, виконує операцію UPDATE у базі даних MySQL/PostgreSQL та повертає статус успішного виконання. Після отримання підтвердження інтерфейс застосунку виводить сповіщення про успішне збереження змін.

Алгоритм процесу "Отримання доступу до розширеної аналітики"

Алгоритм описує логіку взаємодії користувача з платформою при спробі отримати доступ до деталізованої статистики (наприклад, xG-метрики або теплові карти), враховуючи перевірку рівня підписки та прав доступу.

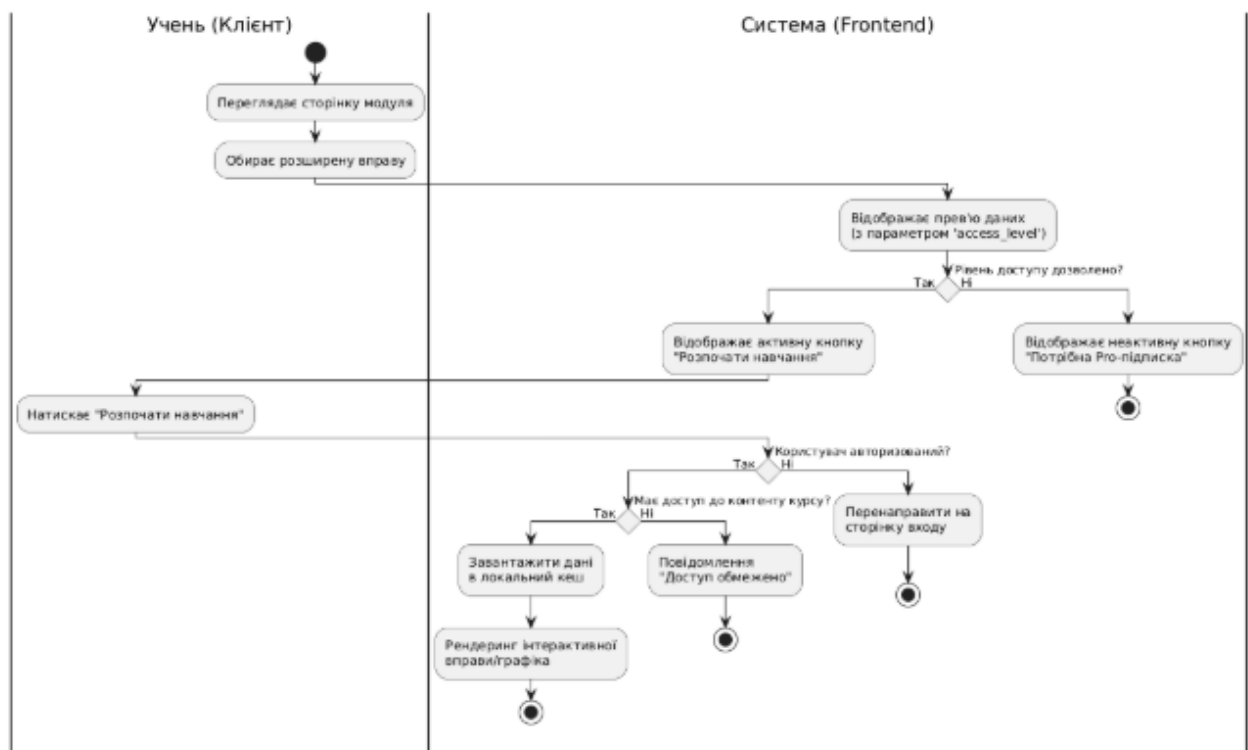


Рис. 2.14 – Діаграма активності "Отримання доступу до розширеного освітнього контенту"

Покроковий опис алгоритму:

1. **Перегляд:** Учень відкриває сторінку курсу або конкретного мовного модуля. Frontend-частина застосунку автоматично зчитує параметр `access_level` для кожного елемента контенту.

2. **Умова доступу:** Якщо навчальний матеріал належить до преміум-сегмента (наприклад, спеціалізовані професійні словники), а користувач має базовий рівень доступу, система відображає неактивну кнопку з пропозицією оновити підписку.
3. **Перевірка авторизації:** При спробі активувати розширений функціонал застосунок перевіряє стан поточної сесії. У разі відсутності авторизації користувач перенаправляється на екран входу.
4. **Захист даних:** Додатково система перевіряє права доступу до специфічних баз даних (наприклад, корпоративних глосаріїв) відповідно до політики RBAC. Якщо доступ обмежений, виводиться відповідне сервісне повідомлення.
5. **Візуалізація:** У разі успішного проходження всіх етапів перевірки, дані завантажуються в локальне сховище пристрою (localStorage або SQLite), після чого активується рендеринг інтерактивних вправ та графіків прогресу.

Алгоритм процесу "Призупинення дашборда Адміністратором"

Цей механізм є ключовим інструментом системної модерації, який забезпечує автоматизоване реагування на публікацію неякісного або недостовірного освітнього контенту.

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				
Змн.	Арк.	№ докум.	Підпис	Дата		56

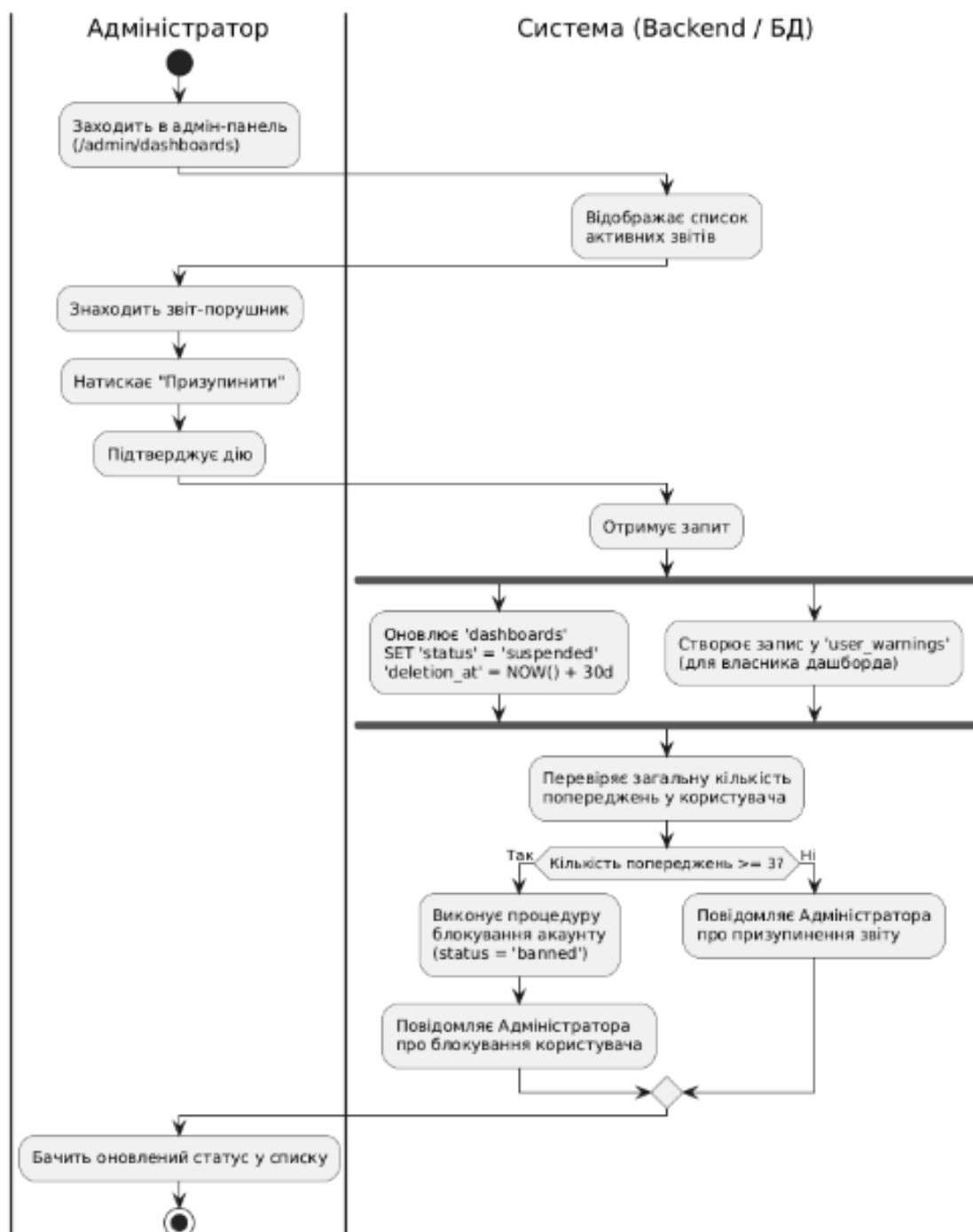


Рис. 2.15 – Діаграма активності " Призупинення навчального модуля Адміністратором"

Покроковий опис алгоритму:

- Ініціалізація:** Адміністратор платформи через спеціалізовану панель керування ідентифікує навчальний модуль, що порушує правила спільноти, та ініціює процедуру його призупинення.
- Паралельне виконання:** Після підтвердження дії Backend-сервер одночасно запускає два потоки: оновлює статус модуля на suspended (з

автоматичним плануванням видалення через 30 днів для очищення бази даних) та формує запис у таблиці user_warnings для фіксації порушення в історії користувача.

3. **Перевірка рецидивів:** Система автоматично виконує агрегаційний запит до бази даних для підрахунку загальної кількості активних попереджень у автора даного модуля.
4. **Прийняття рішення:** На основі отриманих даних спрацьовує логічне розгалуження. Якщо кількість попереджень досягла або перевищила критичну межу (3 і більше), система ініціює каскадне блокування всього облікового запису користувача та приховує всі його публічні матеріали. У випадку, якщо ліміт не перевищено, обмеження застосовуються виключно до поточного модуля.

Висновки до другого розділу

У другому розділі було виконано комплексне проектування програмної архітектури мобільної інформаційної системи для вивчення іноземних мов. Процес охопив усі етапи: від побудови високорівневої моделі інфраструктури до деталізації алгоритмів обробки лінгвістичних даних та синхронізації прогресу.

Побудовано модель розгортання: Обґрунтовано розподілений хмарний підхід із використанням сучасних PaaS та SaaS рішень (Render, Aiven, Cloudinary). Така конфігурація гарантує відмовостійкість при роботі з мультимедійним контентом, швидку доставку даних через CDN та незалежне масштабування клієнтського застосунку і серверної логіки.

Формалізовано варіанти використання: Визначено ролі акторів (Учень, Модератор, Адміністратор) та розроблено специфікації ключових прецедентів. Особливу увагу приділено логіці формування персоналізованих модулів та механізмам модерації контенту, що забезпечує дотримання освітніх стандартів платформи.

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				58
Змн.	Арк.	№ докум.	Підпис	Дата		

Розроблено об'єктно-орієнтовану структуру: Побудовані діаграми класів ілюструють застосування патернів MVC, Factory та Singleton. Використання архітектури Feature-Sliced Design для клієнтської частини дозволяє ізольовано розвивати навчальні модулі та ігрові механіки, не порушуючи цілісність ядра системи.

Спроектовано гібридну модель даних: Логічна структура бази даних поєднує переваги реляційного підходу (нормалізація до 3НФ для профілів користувачів та статистики) із гнучкістю формату JSON для зберігання динамічних конфігурацій вправ. Це дозволяє системі адаптуватися до нових типів контенту без складних міграцій схеми БД.

Описано динамічну поведінку системи: За допомогою діаграм активності, послідовності та кооперації деталізовано бізнес-логіку створення контенту та проходження тестів. Візуалізовано хронологію взаємодії між мобільним клієнтом, контролерами та шаром даних у межах REST API архітектури.

Розроблені логічні та структурні моделі повністю задовольняють вимоги технічного завдання. Вони сформували вичерпну базу для переходу до етапу фізичної реалізації програмного продукту — написання коду, налаштування серверного середовища та розгортання бази даних, що буде детально описано у наступному розділі.

		Сокульський О.І.			ДУ «Житомирська політехніка». 26.121.23.000 - ПЗ	Арк.
		Вакалюк Т.А.				59
Змн.	Арк.	№ докум.	Підпис	Дата		